

ARM Code Optimization for Image Converting with Keil MDK tool

8조 박대호 유지원 최은지 김선영

CONTENTS

1. Project 개요
2. 3가지 이미지 변환 함수의 동작 방식과 block diagram
3. 3가지 이미지 변환 함수의 코드
4. ARM 코드 최적화를 위한 C코드 및 ARM 코드
5. 최적의 ARM 코드 탐색
6. 3번과 4번의 사이의 ARM 코드 성능 차이 분석
7. 결과물들
8. 프로젝트 수행일지
9. Troubleshooting

1. Project 개요

1. Keli MDK 활용 >> ARM 기반 임베디드 시스템에서의 이미지 처리 (Image Processing) 및 변환 (Image Converting) 기능 구현 & 해당 기능들의 성능 최적화 & 비교 분석 수행

- 1. 32bit RGBA 이미지 데이터로부터 Red 채널의 값이 128 이상인 픽셀의 개수를 세는 알고리즘 구현 >> 조건 기반 이미지 분석 기능 수행
- 2. 원본 32bit RGBA 이미지를 24bit Negative RGB 이미지로 변환하는 알고리즘 구현
- 3. 동일한 원본 이미지를 16bit Grayscale 이미지로 변환하는 알고리즘 구현
- 1&2&3 : C언어와 어셈블리어로 구현 >> 각 함수의 핵심 연산은 어셈블리어를 통해 성능 향상을 고려하여 작성

1. Project 개요

2. 다양한 ARM 최적화 기법 적용

- 루프 언롤링, 조건 분기 제거, 메모리 접근 방식 개선, 레지스터 활용 etc...)
- 메모리 재배치를 통한 변환 효율 극대화 방법 실습

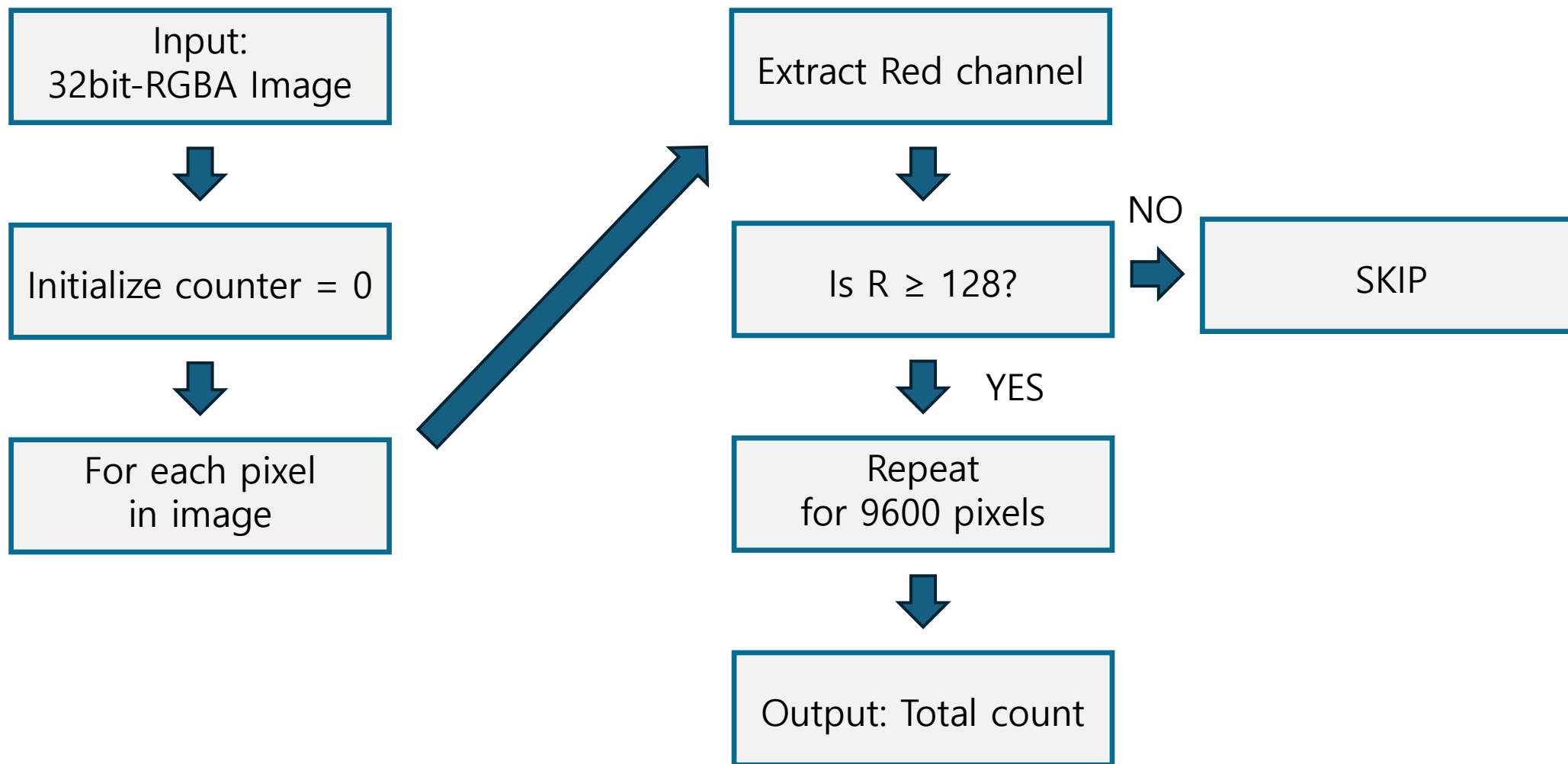
3. 초기 버전의 ARM 코드와 최적화된 ARM 코드 간의 성능 비교

- 코드 구조가 실제 성능에 미치는 영향 평가

**** 본 Project를 통해 이미지 처리 알고리즘을 구현해 봄으로써, 임베디드 시스템에서의 최적화 사고력 및 실전 디버깅 역량을 종합적으로 배양 가능**

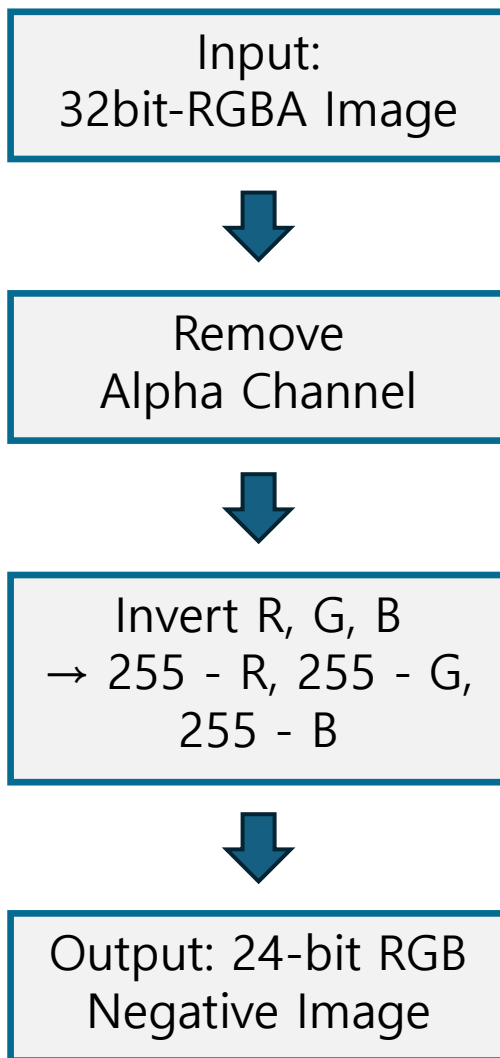
2. 3가지 이미지 변환 함수의 동작 방식과 block diagram

▪ Subproject#1: Red Pixel Count



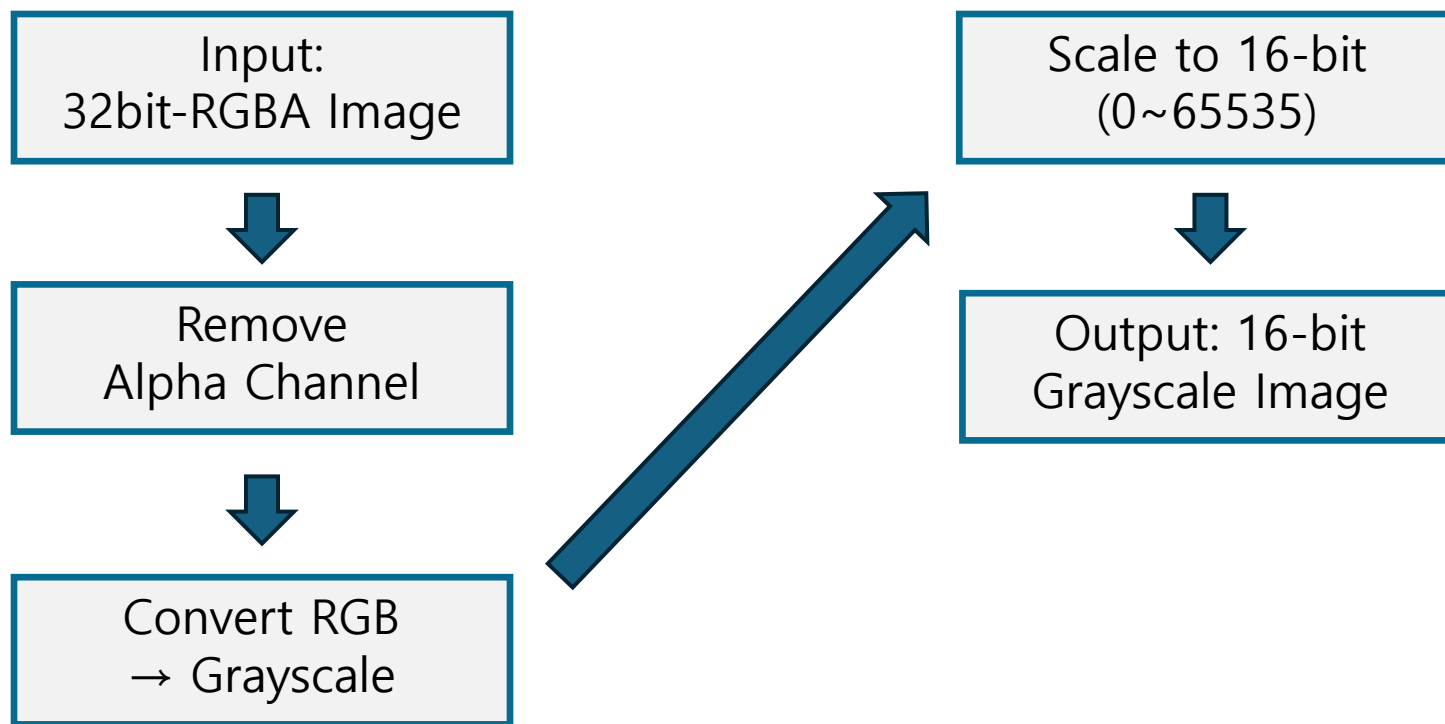
2. 3가지 이미지 변환 함수의 동작 방식과 block diagram

- Subproject#2: Image Negative



2. 3가지 이미지 변환 함수의 동작 방식과 block diagram

▪ Subproject#3: Grayscale



3. 3가지 이미지 변환 함수의 코드

- Subproject#1: Assem.s

```
performance test_1.sct  assem.s
1      AREA Reset, CODE, READONLY
2      EXPORT Red_Pixel_Count
3
4      Red_Pixel_Count
5      LDR    r0, =0x40000036
6      MOV    r1, #9600      ; 총 픽셀 수
7      MOV    r2, #0         ; count
8      MOV    r3, #0         ; 현재 인덱스
9
10     loop
11         CMP    r3, r1      ; r3과 r1 비교_모든 픽셀 검사 여부 확인
12         BEQ    done        ; 같아지면 loop 종료
13
14         LDRB   r4, [r0]; r4_ 현재 픽셀의 red 값
15         CMP    r4, #128; red 값이 128 이상 인지 확인
16         BLT    skip        ; 128 미만이면 count 증가 없이 skip으로 branch
17         ADD    r2, r2, #1 ; 조건 만족 시 r2 증가_count +1
18
19     skip
20         ADD    r0, r0, #4 ; 다음 픽셀로 이동_RGBA=4B
21         ADD    r3, r3, #1 ; 인덱스 증가(다음 loop 준비)
22         B      loop        ; loop 반복
23
24     done
25         MOV    r0, r2      ; count를 r0에 복사
26         BX     lr          ; 호출한 함수로 복귀
27
28     END
```


3. 3가지 이미지 변환 함수의 코드

■ Subproject#1: 결과

Register	Value
Current	
R0	0x000012C3
R1	0x00002580
R2	0x000012C3
R3	0x00002580
R4	0x00000020
R5	0x00000000
R6	0x00000000
R7	0x00000000
R8	0x00000000
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13 (SP)	0x00000000
R14 (LR)	0x00000000
R15 (PC)	0x00000038
CPSR	0x600000D3
SPSR	0x00000000

R0 register 값 = red 값이 128 이상인 pixel의 수
: 0x12C3=4803

3. 3가지 이미지 변환 함수의 코드

■ Subproject#1: 결과 및 설명 (함수 수행 전 후의 memory map 정보)

Address: 0x41004B00	Memory 1 Address: 0x40000036
0x41004B00: 5C 23 1B B2 58 B5 DF B3 6D 40 F7 1C 92 9E C3 F1 17 D2	0x40000036: 78 5E 5C BD 77 73 23 59 92 EC 1B 29 21 08 B2 44 77 8F 58 71 67 EF B5 77 FF 7B
0x41004B12: 6D 86 D0 D1 5D C5 B6 6D C6 C7 C7 B6 18 B4 14 34 47 6C	0x40000050: DF FF F3 3C B3 B5 DD 9D 6D 59 82 02 40 AA 67 3F F7 08 80 B3 1C EB A9 2A 92 00
0x41004B24: C4 BB EF 5D FE 4D 6D E5 EB 6D 66 97 F5 03 D6 53 D0 CB	0x40000064: 32 4F 9E 13 C2 C3 C3 A3 F9 F0 F1 B8 B5 7D 17 73 33 C5 D2 75 D1 B6 6D 6C EB 35
0x41004B36: DF 79 9A 7F 6B F7 FD 31 E3 5A 44 5B 6C 37 4B 5B 6B F6	0x40000084: 86 B5 89 DD D0 C7 DC 4F D1 F6 4D 34 5D 1B CD BA C5 D6 6C B1 B6 5B F0 B5 6D 5B
0x41004B48: 3E E7 D8 AC 30 D1 7F 7A 13 31 F3 BB D6 FB BB EE DF 48	0x4000009E: 74 31 C6 DA 2C D1 C7 16 CD 32 C7 D2 AE B1 B6 5D B4 CD 18 6B DB 46 B4 E7 8B CB
0x41004B5A: D5 6D D3 65 EB 4E E7 63 6E F9 D8 6D 9E 29 6D 3D FA EB	0x400000B8: 14 D1 B6 31 34 6D B4 BC 47 D7 C6 D4 6C 31 AD 4B C4 14 31 F4 BB D8 DA 55 EF D7
0x41004B6C: 7D F9 6F 63 E0 3E F3 78 FE 6D 7D 9E FB 9E 6B FA 8B B2	0x400000D2: 44 17 5D D3 46 34 FE 37 EF 17 4D A3 CF 8B 6D 0B 3E 7A E5 9F 4D 13 EB BA 06 7F
0x41004B7E: CF 5D 3D 5E 87 7F 16 B5 8D E3 A3 3D BB 07 F7 A6 EB 6C	0x400000EC: 6D DB 26 BA 66 8B 79 E5 97 9A D8 A2 F5 F5 F0 FB 03 BF B1 C6 6E 74 31 4D 53 74
0x41004B90: 88 72 6C 37 8E 75 18 77 F1 4F 3F 3C 17 1E 4E 0F E2 E9	0x40000106: 0D 9F D0 47 B3 F0 CB 4D CC BA DF 8E 5F 89 79 5A A3 69 9A 98 E7 59 7F AE 2B 3F
0x41004BA2: 38 DA 29 5D 75 6F 89 4F 59 11 98 5B B9 4C 69 36 9E BA	0x40000120: 6B 22 86 46 F7 16 CD 18 FD CA 05 6C 31 F3 DE 4D E3 F5 B8 2E 5A 98 B1 3F 44 2C
0x41004BB4: 9C 2E F7 DE 79 98 F3 7F C7 7E 0F 78 5D D1 2B 5B 7A B6	0x4000013A: 73 CC 5B CA DC 46 6C 7D 44 37 37 D1 6D 11 4B 13 D1 2F 5B CC CD 12 6B B7 45 D3
0x41004BC6: 21 6B E6 8B BE 4F B9 62 59 EC AA 45 2E AD A2 FA 9A DF	0x40000154: F6 B1 6C 8D 3E 87 7B E5 E7 DB BC C4 D8 36 B1 2C AC 65 13 C3 30 C4 D6 4E D1 6C
0x41004BD8: 69 BE E9 8F 0D 3F 43 7D F9 7E DE 9A D0 AE 7F 98 0E B6	0x4000016E: 9D 6F 7F DA A2 E3 7A AF B3 9E 13 5F D3 18 31 0E 6D CC F3 12 03 6B BB B6 5A 1F
0x41004BEA: 04 56 6C 9C 9F 67 B9 EC E7 FC AD CF 69 B7 8D 0F D0 63	0x40000188: D6 9A 35 9A FB 26 FA 76 8B 66 6C B5 EE 5B DF EA DF 5D 7F F0 48 F4 55 EF D5 0F
0x41004BFC: B1 5C 96 6B AE F6 B9 4D 63 43 1B 80 3C BB FF 1A 58 5F	0x400001A2: 4D 6C 6D C3 6A 45 D3 2C FA AC 65 19 74 23 EB 36 C7 D0 4E D1 C6 E6 E7 B2 69 B1
0x41004C0E: D1 7B 31 9B BF FE 66 3B 9F 31 A1 F4 DB 7C 35 DF 2E DA	0x400001BC: 63 6D 87 D0 6E 60 3D 58 F9 B6 D5 B3 D8 96 88 A1 6D 62 DB B1 9E 48 2C D7 29 BA
0x41004C20: 48 2B 64 36 6F 17 6D 1D C6 9D 2E 78 7C BE 07 1B E2 DF	0x400001D6: AE 8B 6D 6B F4 7A 3D EB C6 F7 FA FE B9 2F EB A4 F5 E7 7D F8 6F 99 F9 BD 56 FB
0x41004C32: 57 4F D8 E3 5E AC D7 B2 79 DF 4D 75 11 0C 9B 56 D8 AB	0x400001F0: 6F DE 96 58 63 89 86 E5 E0 5E A3 0F 3E 48 7B 76 F3 73 E1 96 78 5F BE C7 FE B8
0x41004C44: 98 5D 6B 31 DB FC 3C 9E 4F B5 9A 9C 17 8A 9D 7A 64 53	0x40000204: F2 8A 6D 8B 31 06 7D DF 9F CB 9E F2 B3 F6 FB D7 F7 79 9E A1 E7 E5 6B CD 7D BE
0x41004C56: EB F6 FC 73 DE 9D F7 F8 37 E2 1B D9 7B D9 79 26 34 8F	0x40000224: FA 1E FA BE 8B A6 F5 F5 B2 75 EB 4B CF 30 CF 43 5D 0B DF E3 3D 16 36 42 5E D3
0x41004C68: 7E DA D6 5B 6B 79 F8 F6 69 CE 5B 93 7C EF 7D E8 A7 71	0x4000023E: BA 82 87 58 B7 CD 7F 6E 5C E3 16 B1 AD B1 B5 87 68 A2 8D 69 99 A2 E3 D8 2C 3C
0x41004C7A: 1D 71 61 75 B1 DB B0 D6 AF D1 2D 97 3A A6 F8 60 88 6E	0x40000258: A3 41 F7 CB 3D F2 99 6D BB 8F 75 F1 07 D7 FA B1 F7 B5 BF B5 A6 5E 87 B6 EB 62
0x41004C8C: 8A B8 81 39 F5 1B E6 9E CB A6 E4 BC 9F E7 91 55 73 2C	0x40000272: E5 01 6C BE 06 6D 88 75 8E 36 72 BF B1 6E 6C 78 2D 23 37 DC 46 D3 8E B1 C4 A4
0x41004C9E: 16 7A 61 B9 18 46 1C 39 35 70 49 75 CA C6 9C E2 7F EC	0x40000284: 75 EE 87 8B 1B BB 38 1C 77 B1 3F 34 F8 F8 14 4F BB 0F B1 3F 3C C5 61 3C C6
0x41004CB0: FA E3 F9 C4 C4 CE 35 3B A2 2E 16 9B E1 C6 2E BA C6 E2	0x400002A6: FE 61 17 8F BB 31 1E 0F FB 18 4E BB 38 EC 0F 71 1A 9E E2 B8 3F C5 E9 70 8A 71
0x41004CC2: F8 BD AE 17 67 95 E8 BD B7 5E 1E CF FB 71 F7 BC F7 02	0x400002C0: 38 C4 D0 8D DA BF FB F6 29 BA E1 1A 5D 3F 6B FF 75 ED 51 7B 6F D0 91 7B 89 A6
0x41004CD4: 04 EB BC F3 BE 78 FC 16 3F B0 10 3C F3 E7 87 B7 9F 4C	0x400002DA: FB 35 4F EC 35 A2 59 62 59 CF 11 CD 35 B6 98 A3 61 F9 5B 56 A5 D1 B9 9F AF 53
0x41004CE6: BA 31 D3 B7 E2 E2 E2 F1 F3 16 C1 C0 B6 8C 7A 7E 33 BD	0x400002F4: 4C D3 1A 97 69 17 CF D7 36 5E E7 3E 9E 97 88 E5 BA C6 BC F1 9C BB D8 96 2E 96
0x41004CF8: 38 85 E3 23 0F F8 6D 49 B1 7F 70 16 DC 7A 0A 95 65 71	0x4000030E: F3 D5 F7 36 37 71 DE CE F1 72 79 8D 98 C7 98 F8 DD 85 F3 D8 C7 71 7F 88 A7 C7
0x41004D0A: 72 B7 B0 B9 3A 0F 20 5E 3A E6 D8 71 3B 22 57 B5 EE 59	0x40000328: C7 F8 97 1F 7E 8C 0F 4F 0F 71 1C BB 78 38 8C 11 5D C4 D0 75 D1 B7 43 4C 2B BF
0x41004D1C: E5 73 0C 9F CE D8 78 29 6E B4 60 8C 31 8A 30 5D C7 14	0x40000342: CB 3E 5B 62 5D AF 7A F6 3C B3 B6 ED A3 89 21 B6 DC BB 6B BB F8 79 E6 81 EA A3
0x41004D2E: DA F7 A1 F3 07 84 BF 32 FD E9 E7 86 1C D6 25 79 8C 5E	0x4000035C: 8B DB 78 D0 BE 6D 3B EF 4F F6 43 C3 B9 90 DD D8 62 C9 E7 C8 59 98 79 1F EC 91
0x41004D40: 81 11 BE E3 18 D7 02 22 D9 3A 40 A8 A4 F0 05 6C 82 38	0x40000376: F6 DB AA 7B 5D E6 45 E7 6A 8D 2E B6 66 95 AD E9 DA 21 A2 99 A3 1B FA 68 B6 36
0x41004D52: 3E 66 A8 B3 80 0C BF 9F 37 15 0F E8 87 7C 87 F4 7C 1F	0x40000390: 9A 79 8D AE DF 7B AF B6 69 23 D6 D0 BE D7 73 F7 E9 8E 2D F7 8F EC A5 EC OD 9F
0x41004D64: EC 0F 9C 8D 87 E3 FB 8C 16 79 86 68 FD 46 43 DE 3C 26	0x400003AA: 3F E9 3F 9D 75 ED 43 3E 7E D2 7D AC B9 EF F9 59 BD F6 7E 6E D9 7F DE D3 7A FD
0x41004D76: B5 C4 6D 8F E0 AF D8 C4 F9 DA B1 1E D8 9F A7 63 C6 68	0x400003C4: 9A 7B 2F D6 D0 FE E6 8C AE 73 AC 2B 7F F2 5E E7 98 65 CB B1 0E 2C 9F 2C B6 CF
0x41004D88: FC F9 EA C8 E4 5F F4 2D 5D 87 A3 A2 E2 09 0B AF BC F9	0x400003DE: 96 D6 04 C3 B7 D9 56 E8 AD F3 6C AC 5B 34 9C BB C6 7B 9F BD 50 76 67 DB D6 98
0x41004D9A: A4 8E 39 27 C4 C0 A1 84 1D 1B 8C 7C C9 5C 91 16 CF 9D	0x400003F8: B9 36 EE 33 EC 07 F0 19 57 F5 BE D6 FC 0E 1F B1 AD 53 74 BD CF 38 96 BF 69 86
0x41004DAC: C0 5C FE 21 AD 36 A6 92 11 D1 0E 8F 89 09 CA 01 E6 9D	0x40000412: 58 58 B7 8E 4D 8E 8D E4 9E 79 0F EE B5 7C D0 1C 0B 96 63 5D A2 99 B1 89 EC 81
	0x4000042C: 5C 23 7D 7F 96 91 98 D6 6B 6C EB 4B AE C5 1C 9B F6 2E D7 C5 B9 1A 74 CD 4D 83
	0x40000446: 05 5A 63 5E 16 1D 43 5F 53 27 1B A6 EB 9E B0 FD 73 CC 3C 13 F6 5A BB EA OC C8
	0x40000460: FF 6C 11 FD 1A D1 E1 98 5B 1C D6 0E 5F 21 FB 1B D1 35 11 4B 7B 89 2E FD 31 D7
	0x4000047A: D5 76 98 DE 97 B3 BF E5 19 E7 FE B0 29 7A 66 98 BE F4 3B 6D 74 DA 9F 7A 36 F8
	0x40000494: 31 D9 10 1E A1 F7 5B DB F4 B1 CE 57 DB 47 19 D6 7C 13 1D CC 35 7D 6C AB DF C3
	0x400004AE: 97 CA 2E 71 4D EC DA DB FE E3 48 B1 F6 3C 2B FE 24 7B 64 8E A5 C7 36 B5 B1 E2

- 메모리에 쓰기 연산 수행 X, 읽기만 수행 O
- 함수 수행 전후의 메모리 내용 동일하게 유지 & red 값이 128 이상인 픽셀의 개수는 연산 도중 레지스터에서만 처리됨
- 최종 결과는 r0 레지스터에 저장된 상태로 함수가 종료됨

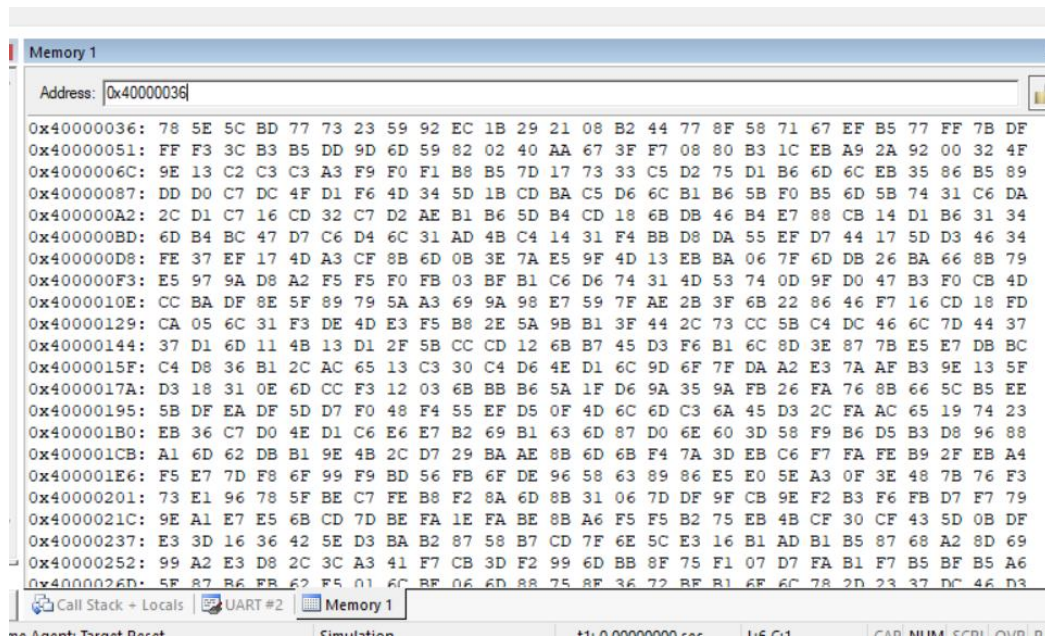
3. 3가지 이미지 변환 함수의 코드

■ Subproject#2: Assem.s

```
1      AREA    Reset, CODE, READONLY
2      ENTRY
3      EXPORT  rgb_negative
4
5      rgb_negative
6
7      LDR     r0, =0x40000036      ; input image start address (RGBA)
8      LDR     r1, =0x20000000      ; output image start address (RGB)
9      MOV     r2, #9600            ; total 9,600 pixels to process
10
11     loop
12         ; Read and invert R
13         LDRB  r3, [r0], #1        ; r3 = R, r0 += 1
14         RSBS  r3, r3, #255        ; r3 = 255 - R
15
16         ; Read and invert G
17         LDRB  r4, [r0], #1        ; r4 = G, r0 += 1
18         RSBS  r4, r4, #255        ; r4 = 255 - G
19
20         ; Read and invert B
21         LDRB  r5, [r0], #1        ; r5 = B, r0 += 1
22         RSBS  r5, r5, #255        ; r5 = 255 - B
23
24         ; Skip Alpha (just move pointer 1 byte forward)
25         ADD   r0, r0, #1
26
27         ; Store inverted RGB values (3 bytes per pixel)
28         STRB  r3, [r1], #1        ; store R
29         STRB  r4, [r1], #1        ; store G
30         STRB  r5, [r1], #1        ; store B
31
32         ; Loop control
33         SUBS  r2, r2, #1
34         BNE   loop
35
36         BX    lr                  ; return
37
38     END
```

3. 3가지 이미지 변환 함수의 코드

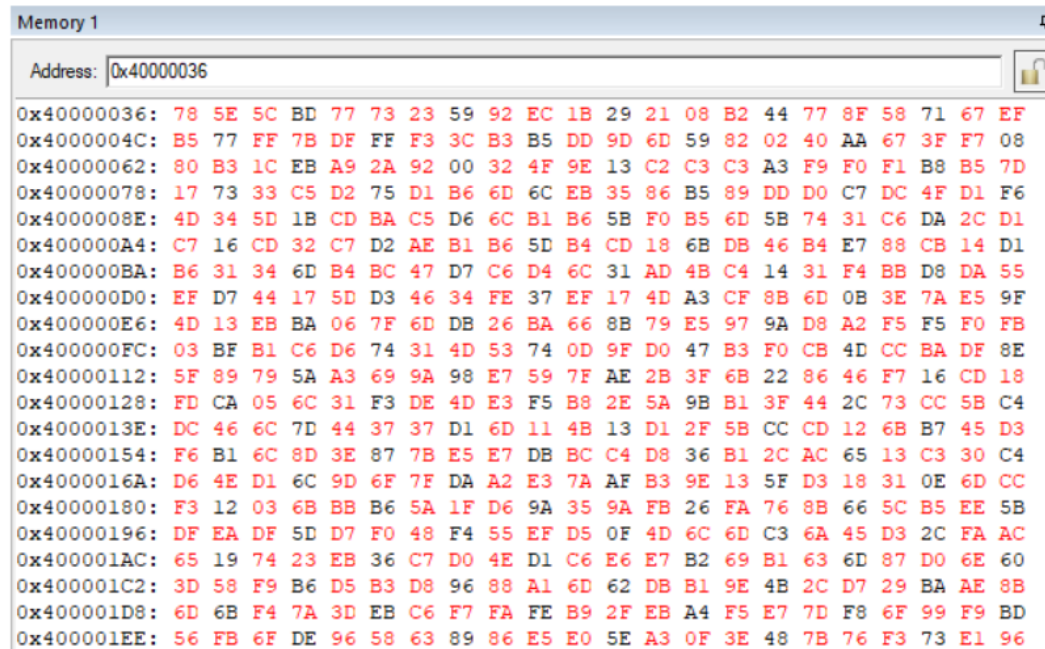
- Subproject#2: 결과 및 설명 (함수 수행 전 후의 memory map 정보)



Memory 1

Address: 0x40000036

```
0x40000036: 78 5E 5C BD 77 73 23 59 92 EC 1B 29 21 08 B2 44 77 8F 58 71 67 EF B5 77 FF 7B DF
0x40000051: FF F3 3C B3 B5 DD 9D 6D 59 82 02 40 AA 67 3F F7 08 80 B3 1C EB A9 2A 92 00 32 4F
0x4000006C: 9E 13 C2 C3 C3 A3 F9 F0 F1 B8 B5 7D 17 73 33 C5 D2 75 D1 B6 6D 6C EB 35 86 B5 89
0x40000087: DD D0 C7 DC 4F D1 F6 4D 34 5D 1B CD BA C5 D6 6C B1 B6 5B F0 B5 6D 5B 74 31 C6 DA
0x400000A2: 2C D1 C7 16 CD 32 C7 D2 AE B1 B6 5D B4 CD 18 6B DB 46 B4 E7 88 CB 14 D1 B6 31 34
0x400000BD: 6D B4 BC 47 D7 C6 D4 6C 31 AD 4B C4 14 31 F4 BB D8 DA 55 EF D7 44 17 5D D3 46 34
0x400000D8: FE 37 EF 17 4D A3 CF 8B 6D 0B 3E 7A E5 9F 4D 13 EB BA 06 7F 6D DB 26 BA 66 8B 79
0x400000F3: E5 97 9A D8 A2 F5 F5 F0 FB 03 BF B1 C6 D6 74 31 4D 53 74 0D 9F D0 47 B3 F0 CB 4D
0x4000010E: CC BA DF 8E 5F 89 79 5A A3 69 9A 98 E7 59 7F AE 2B 3F 6B 22 86 46 F7 16 CD 18 FD
0x40000129: CA 05 6C 31 F3 DE 4D E3 F5 B8 2E 5A 9B B1 3F 44 2C 73 CC 5B C4 DC 46 6C 7D 44 37
0x40000144: 37 D1 6D 11 4B 13 D1 2F 5B CC CD 12 6B B7 45 D3 F6 B1 6C 8D 3E 87 7B E5 E7 DB BC
0x4000015F: C4 D8 36 B1 2C AC 65 13 C3 30 C4 D6 4E D1 6C 9D 6F 7F DA A2 E3 7A AF B3 9E 13 5F
0x4000017A: D3 18 31 0E 6D CC F3 12 03 6B BB B6 5A 1F D6 9A 35 9A FB 26 FA 76 8B 66 5C B5 EE
0x40000195: 5B DF EA DF 5D D7 F0 48 F4 55 EF D5 0F 4D 6C 6D C3 6A 45 D3 2C FA AC 65 19 74 23
0x400001B0: EB 36 C7 D0 4E D1 C6 E6 E7 B2 69 B1 63 6D 87 D0 6E 60 3D 58 F9 B6 D5 B3 D8 96 88
0x400001CB: A1 6D 62 DB B1 9E 4B 2C D7 29 BA AE 8B 6D 6B F4 7A 3D EB C6 F7 FA FE B9 2F EB A4
0x400001E6: F5 E7 7D F8 6F 99 F9 BD 56 FB 6F DE 96 58 63 89 86 E5 E0 5E A3 0F 3E 48 7B 76 F3
0x40000201: 73 E1 96 78 5F BE C7 FE B8 F2 8A 6D 8B 31 06 7D DF 9F CB 9E F2 B3 F6 FB D7 F7 79
0x4000021C: 9E A1 E7 E5 6B CD 7D BE FA 1E FA BE 8B A6 F5 F5 B2 75 EB 4B CF 30 CF 43 5D 0B DF
0x40000237: E3 3D 16 36 42 5E D3 BA B2 87 58 B7 CD 7F 6E 5C E3 16 B1 AD B1 B5 87 68 A2 8D 69
0x40000252: 99 A2 E3 D8 2C 3C A3 41 F7 CB 3D F2 99 6D BB 8F 75 F1 07 D7 FA B1 F7 B5 BF B5 A6
0x4000026F: 5F 87 86 FB 62 F5 01 6C BF 06 6D 88 75 8F 36 72 BF B1 6F 6C 78 2D 23 37 DC 46 D3
```



Memory 1

Address: 0x40000036

```
0x40000036: 78 5E 5C BD 77 73 23 59 92 EC 1B 29 21 08 B2 44 77 8F 58 71 67 EF
0x4000004C: B5 77 FF 7B DF FF F3 3C B3 B5 DD 9D 6D 59 82 02 40 AA 67 3F F7 08
0x40000062: 80 B3 1C EB A9 2A 92 00 32 4F 9E 13 C2 C3 C3 A3 F9 F0 F1 B8 B5 7D
0x40000078: 17 73 33 C5 D2 75 D1 B6 6D 6C EB 35 86 B5 89 DD D0 C7 DC 4F D1 F6
0x4000008E: 4D 34 5D 1B CD BA C5 D6 6C B1 B6 5B F0 B5 6D 5B 74 31 C6 DA 2C D1
0x400000A4: C7 16 CD 32 C7 D2 AE B1 B6 5D B4 CD 18 6B DB 46 B4 E7 88 CB 14 D1
0x400000BA: B6 31 34 6D B4 BC 47 D7 C6 D4 6C 31 AD 4B C4 14 31 F4 BB D8 DA 55
0x400000D0: EF D7 44 17 5D D3 46 34 FE 37 EF 17 4D A3 CF 8B 6D 0B 3E 7A E5 9F
0x400000E6: 4D 13 EB BA 06 7F 6D DB 26 BA 66 8B 79 E5 97 9A D8 A2 F5 F5 F0 FB
0x400000FC: 03 BF B1 C6 D6 74 31 4D 53 74 0D 9F D0 47 B3 F0 CB 4D CC BA DF 8E
0x40000112: 5F 89 79 5A A3 69 9A 98 E7 59 7F AE 2B 3F 6B 22 86 46 F7 16 CD 18
0x40000128: FD CA 05 6C 31 F3 DE 4D E3 F5 B8 2E 5A 9B B1 3F 44 2C 73 CC 5B C4
0x4000013E: DC 46 6C 7D 44 37 37 D1 6D 11 4B 13 D1 2F 5B CC CD 12 6B B7 45 D3
0x40000154: F6 B1 6C 8D 3E 87 7B E5 E7 DB BC C4 D8 36 B1 2C AC 65 13 C3 30 C4
0x4000016A: D6 4E D1 6C 9D 6F 7F DA A2 E3 7A AF B3 9E 13 5F D3 18 31 0E 6D CC
0x40000180: F3 12 03 6B BB B6 5A 1F D6 9A 35 9A FB 26 FA 76 8B 66 5C B5 EE 5B
0x40000196: DF EA DF 5D D7 F0 48 F4 55 EF D5 0F 4D 6C 6D C3 6A 45 D3 2C FA AC
0x400001AC: 65 19 74 23 EB 36 C7 D0 4E D1 C6 E6 E7 B2 69 B1 63 6D 87 D0 6E 60
0x400001C2: 3D 58 F9 B6 D5 B3 D8 96 88 A1 6D 62 DB B1 9E 4B 2C D7 29 BA AE 8B
0x400001D8: 6D 6B F4 7A 3D EB C6 F7 FA FE B9 2F EB A4 F5 E7 7D F8 6F 99 F9 BD
0x400001EE: 56 FB 6F DE 96 58 63 89 86 E5 E0 5E A3 0F 3E 48 7B 76 F3 73 E1 96
```


3. 3가지 이미지 변환 함수의 코드

- **Subproject#2: 결과 및 설명 (함수 수행 전 후의 memory map 정보)**

Memory 1																																
Address:		0xE2000000																														
0xE2000000:	87	A1	A3	88	8C	DC	6D	13	E4	DE	F7	4D	88	70	A7	98	10	4A	00	84	20	0C										
0xE20000016:	C3	4C	22	62	92	7D	FD	BF	98	C0	08	7F	4C	E3	56	D5	6D	CD	B0	61	3D	3C										
0xE2000002C:	3C	06	0F	0E	4A	82	E8	CC	3A	2D	2E	49	92	14	CA	79	76	22	2F	23	B0	2E										
0xE20000042:	B2	CB	A2	32	45	3A	93	4E	49	0F	4A	92	8B	CE	39	D3	2E	38	32	CD	38	51										
0xE20000058:	4E	49	4B	32	E7	24	B9	4B	77	34	EB	49	CE	CB	4B	43	B8	39	2B	93	52	B4										
0xE2000006E:	3B	CE	0B	44	25	AA	10	BB	E8	A2	B9	CB	01	10	E8	B2	30	74	92	C1	85	1A										
0xE20000084:	B2	EC	14	F9	80	92	D9	45	99	86	1A	68	27	5D	0A	0F	04	FC	4E	39	29	CE										
0xE2000009A:	B2	AC	F2	60	2F	4C	0F	34	33	45	20	A0	76	86	5C	96	65	18	A6	80	D4	C0										
0xE200000B0:	94	79	B9	08	32	E7	02	FA	93	CE	21	B2	1C	47	D1	A5	4E	C0	BB	8C	33	A4										
0xE200000C6:	23	B9	93	BB	C8	C8	92	EE	B4	2E	D0	A4	32	ED	94	BA	2C	09	93	72	C1	84										
0xE200000DC:	1A	18	43	3B	27	4E	D3	53	EC	3C	CF	29	B1	2E	62	90	80	5D	1C	85	4C	61										
0xE200000F2:	EC	2C	E7	CE	92	33	0C	FC	94	44	A5	E0	29	CA	65	04	05	89	74	A3	4A	11										
0xE20000108:	20	15	20	28	0F	B7	AA	10	2A	B2	93	92	95	BA	2C	05	53	9A	8B	DC	14	38										
0xE2000011E:	2F	B1	39	19	18	96	4E	9C	78	2F	91	C2	A7	06	2A	4C	27	77	5E	92	24	4E										
0xE20000134:	61	D3	28	D6	51	74	92	0B	85	C2	39	08	05	46	D0	14	0A	18	82	90	66	06										
0xE2000014A:	A9	04	90	69	A7	9C	79	1A	1F	5C	F0	C1	84	89	0C	1E	69	87	41	38	01	0D										
0xE20000160:	75	92	CE	F9	82	60	34	61	4C	09	04	08	86	61	18	1A	94	82	41	05	05	41										
0xE20000176:	74	0A	0A	4D	14	B4	30	30	BC	A2	20	1C	C2	C3	BD	A1	45	4D	78	48	32	80										
0xE2000018C:	A3	1C	E9	52	4E	4A	97	5D	72	66	5D	1C	D3	C3	5C	08	34	C2	66	92	44	8A										
0xE200001A2:	0E	F8	05	4E	08	40	4A	59	78	49	14	1A	FE	93	F9	92	77	71	C9	8D	4E	91										
0xE200001B8:	93	D2	DC	C8	B9	2C	71	3B	5B	8A	78	77	E7	C7	E3	88	C0	CB	0E	0F	EB	B0										

3. 3가지 이미지 변환 함수의 코드

■ Subproject#3: Assem.s

```
1      AREA Reset, CODE, READONLY
2      EXPORT gray_scale ; 3*R + 6*G +1*B
3
4      ENTRY
5
6
7
8      gray_scale
9          LDR    r0, start
10         LDR    r1, pixel
11         MOV    r2, #9600
12
13     loop
14         LDRB    r3, [r0], #1      ; R값
15         ADD     r3, r3, r3        ; r3=2*R
16         ADD     r3, r3, r3, LSR #1 ; r3=3*R
17
18         LDRB    r4, [r0], #1      ; G값
19         LSL     r4, r4, #2        ; r4=4*G
20         ADD     r4, r4, r4, LSR #1 ; r4=6*G
21
22         LDRB    r5, [r0], #2      ; B값 (A 사용 X)
23
24         ADD     r6, r3, r4        ; r6= 3*R + 6*G
25         ADD     r7, r6, r5        ; r7= 3*R + 6*G +B
26
27         STRH    r7, [r1], #2      ; r7 값 r1에 저장, r2 2B 증가
28
29         SUBS    r2, r2, #1
30
31         BNE     loop
32     start DCD 0x40000036          ; RGBA 픽셀 데이터 시작 주소
33     pixel DCD 0x20000000          ; Grayscale 결과 저장 주소
34
35     END
```

3. 3가지 이미지 변환 함수의 코드

- Subproject#3: 결과 및 설명 (함수 수행 전 후의 memory map 정보)

Address: 0x20000000	Address: 0x40000000
0x20000000: 6B 12 00 46 01 01 04 7C 21 3A 57 74 24 5C 8C 5A 22 7B 64 BB	0x40000000: 89 50 4E 47 0D 0A 1A 0A 00 00 00 0D 49 48 44 52 00 00 03 C0
0x20000014: EF C8 6D 75 7F 83 49 6A CE D3 45 A3 AB 9D D1 A1 74 A1 A5 67	0x40000014: 00 00 02 80 08 06 00 00 00 53 F9 11 43 00 00 00 01 73 52 47
0x20000028: 83 71 C0 89 79 B7 72 74 8F 6F 6C CA 99 8C 9A 8E 3C 99 78 80	0x40000028: 42 00 AE CE 1C E9 00 00 20 00 49 44 41 54 78 5E 5C BD 77 73
0x2000003C: 78 B7 F2 61 83 40 8B 8C 9F 7E BF 6C 54 86 81 92 D6 85 4F 9B	0x4000003C: 23 59 92 EC 1B 29 21 08 B2 44 77 8F 58 71 67 EF B5 77 FF 7B
0x20000050: 60 84 72 94 7E B1 62 D1 87 5F 83 9E AC 96 73 3D 7B 8A 85 90	0x40000050: DF FF F3 3C B3 B5 DD 9D 6D 59 82 02 40 AA 67 3F F7 08 80 B3
0x20000064: 73 A4 9E 9E 72 96 93 46 90 8F A8 75 55 D7 B0 9E 65 71 AF 94	0x40000064: 1C EB A9 2A 92 00 32 4F 9E 13 C2 C3 C3 A3 F9 F0 F1 B8 B5 7D
0x20000078: DB CC B3 A7 A6 76 9F 5C B3 8E DB 5E AE C8 E9 C2 9C 8C C0 B0	0x40000078: 17 73 33 C5 D2 75 D1 B6 6D 6C EB 35 86 B5 89 DD D0 C7 DC 4F
0x2000008C: 7F 75 29 98 87 6D 63 8E 81 87 9C 97 98 80 D6 82 A6 62 81 98	0x4000008C: D1 F6 4D 34 5D 1B CD BA C5 D6 6C B1 B6 5B F0 B5 6D 5B 74 31
0x200000A0: 52 89 A9 B1 69 78 F4 65 80 9D 69 85 79 40 90 AC 84 DD 85 55	0x400000A0: C6 DA 2C D1 C7 16 CD 32 C7 D2 AE B1 B6 5D B4 CD 18 6B DB 46
0x200000B4: 9F 9F C2 90 5D 6F 82 7D AC 76 73 8E 93 C0 BA 90 96 DF 88 C8	0x400000B4: B4 E7 88 CB 14 D1 B6 31 34 6D B4 BC 47 D7 C6 D4 6C 31 AD 4B
0x200000C8: DD 93 C7 4D 40 62 96 8A 7B 5E 99 C8 7E B1 B5 AA 7C 9C B6 A4	0x400000C8: C4 14 31 F4 BB D8 DA 55 EF D7 44 17 5D D3 46 34 FE 37 EF 17
0x200000DC: 7A C3 83 96 72 CB 9E B1 89 AD 7C A5 8B BD 56 6E 5E 9B A9 A3	0x400000DC: 4D A3 CF 8B 6D 0B 3E 7A E5 9F 4D 13 EB BA 06 7F 6D DB 26 BA
0x200000F0: AC 64 E7 90 A8 98 01 00 00 00 00 00 00 00 92 7B D9 85 80 83	0x400000F0: 66 8B 79 E5 97 9A D8 A2 F5 F5 F0 FB 03 BF B1 C6 D6 74 31 4D
0x20000104: 6F 82 B8 7E 6D 7F BA 50 94 AB 70 92 69 44 3C 51 83 B8 84 7B	0x40000104: 53 74 0D 9F D0 47 B3 F0 CB 4D CC BA DF 8E 5F 89 79 5A A3 69
0x20000118: 88 7D 79 8E 71 5B 84 BA 7F 93 92 57 6A 74 A9 D2 7A 47 59 BB	0x40000118: 9A 98 E7 59 7F AE 2B 3F 6B 22 86 46 F7 16 CD 18 FD CA 05 6C
0x2000012C: 4F EC 9F 8D 84 75 41 1D 71 6B 81 CC 63 89 6F CA E7 8C E0 AE	0x4000012C: 31 F3 DE 4D E3 F5 B8 2E 5A 9B B1 3F 44 2C 73 CC 5B C4 DC 46
0x20000140: 8C A1 CE 8F 5F B6 86 87 7E 96 6D 6C 41 3C 7A 72 7E 86 B9 AC	0x40000140: 6C 7D 44 37 37 D1 6D 11 4B 13 D1 2F 5B CC CD 12 6B B7 45 D3
0x20000154: 8D 72 C7 88 66 99 8E A0 C5 6B 79 B2 C9 52 96 9B 8E D4 B0 88	0x40000154: F6 B1 6C 8D 3E 87 7B E5 E7 DB BC C4 D8 36 B1 2C AC 65 13 C3
0x20000168: A4 CC 8F B4 8A 9D 7C 92 A0 7F 7D 7A 90 B4 A2 8D B0 7B 9B A5	0x40000168: 30 C4 D6 4E D1 6C 9D 6F 7F DA A2 E3 7A AF B3 9E 13 5F D3 18
0x2000017C: A2 8E 9C B7 A6 A6 82 7A 75 A8 B7 91 75 9A A4 6C 6C B4 C9 9F	

3. 3가지 이미지 변환 함수의 코드

- Subproject#3: 결과 및 설명 (함수 수행 전 후의 memory map 정보)

Address: 0x200257A0	Register	Value
0x200257A0: BC 6B 24 64 59 BA B7 43	Current	
0x200257A8: 7A 73 9A 5B 8B E7 9D 90	R0	0x40000037
0x200257B0: C9 60 74 CC 9B A5 9A 83	R1	0x41000001
0x200257B8: 84 4D 76 70 C6 7B 3C 94	R2	0x41002580
0x200257C0: 94 81 98 3D 81 7E 95 9D	R3	0x41004800
0x200257C8: 82 DC 6F 3C B7 84 7E 77	R4	0x00002580
0x200257D0: 63 B7 80 6C 89 A6 33 7F	R5	0x00000078
0x200257D8: 69 AD 8A B4 2A 7E B2 72	R6	0x00000000
0x200257E0: 4B 9E 91 45 6A 60 92 7D	R7	0x00000000
0x200257E8: 4B A2 77 80 7E B0 84 8D	R8	0x00000000
0x200257F0: AB A2 AE 94 81 92 95 49	R9	0x00000000
0x200257F8: 2E 5F 8E 94 B3 1C 4A 6A	R10	0x00000000
0x20025800: 00 00 00 00 00 00 00 00	R11	0x00000000
0x20025808: 00 00 00 00 00 00 00 00	R12	0x00000000
	R13 (SP)	0x00000000
	R14 (LR)	0x00000004
	R15 (PC)	0x00000024
	CPSR	0x000000D3
	SPSR	0x00000000

- RGB(A) 형식의 픽셀 데이터를 입력으로 받아 각 픽셀의 회색조 값을 계산 후 메모리에 저장
- 각 픽셀은 R, G, B, A 순서로 구성, 이 중 A 값은 사용하지 않고 R, G, B 값만 활용
- 각 픽셀에 대해 회색조 변환 공식 3 곱하기 R 더하기 6 곱하기 G 더하기 B 적용

4. ARM 코드 최적화를 위한 C코드 및 ARM 코드

- 이미지 hex 파일의 색상 채널을 메모리에 재배치하는 방식

- 플래너 방식 : 색상 채널을 분리된 영역에 저장하는 방식

- > 특정 채널만 처리하거나 압축할 때 유리함
 - > but 메모리 접근이 복잡 & 병렬처리 hard

- 인터리브드 방식 : 색상 채널을 한 픽셀 단위로 연속으로 저장하는 방식

- > 메모리 접근이 직관적 & 대부분의 디스플레이 장치와 호환됨
 - > but 채널 분리 hard

- 이미지 hex 파일을 alpha 채널 제거 후 메모리에 재배치 >> 이미지 변환 함수를 동작 시키고자 함

- RGB 채널을 인터리브드 방식으로 재배치하면 특이점이 생김
 - Alpha 채널을 제거 후 인터리브드 방식으로 relocation

- > 각 픽셀의 크기가 4byte -> 3byte 로 줄어들게 됨

- > but 연산의 정확도에는 영향 X, 따라서 성능 개선 미미함

- (ARM architecture은 4byte의 align access를 선호하므로, 3byte 단위로 접근할 시 메모리 접근에서의 penalty가 발생 가능)

- 실제로 RGB를 인터리브드 방식으로 메모리에 재배치하여 실험 수행한 결과 >> 성능 개선 저조 & 오류 발생 >> 따라서 ARM 코드 최적화에 중점을 뒀다 메모리 재배치는 모두 플래너 방식으로 진행

4. ARM 코드 최적화를 위한 C코드 및 ARM 코드

■ Subproject#1: Assem.s

```
1      AREA Reset, CODE, READONLY
2      EXPORT start
3
4      start
5          BL relocate
6          BL Red_Pixel_Count_Re
7
8      relocate
9          LDR    r0, =0x40000036    ; r0 = RGBA 데이터 시작 주소
10         LDR    r1, =0x41000000    ; r1 = R값 저장할 목적지 주소
11         LDR    r2, =0x41002580    ; r2 = G값 저장할 목적지 주소
12         LDR    r3, =0x41004B00    ; r3 = B값 저장할 목적지 주소
13         MOV    r4, #9600          ; r4 = 픽셀 수
14
15      loop_relocate
16          LDRB   r5, [r0], #1        ;
17          STRB   r5, [r1], #1        ; R 저장
18
19          LDRB   r5, [r0], #1        ;
20          STRB   r5, [r2], #1        ; G 저장
21
22          LDRB   r5, [r0], #1        ;
23          STRB   r5, [r3], #1        ; B 저장
24
25          ADD    r0, r0, #1          ; A 무시
26
27          SUBS   r4, r4, #1
28
29          BNE    loop_relocate
30          BX     lr                ; 재정렬 끝
31
32
33
34      Red_Pixel_Count_Re
35          LDR    r0, =0x41000000
36          MOV    r1, #9600          ; r1: 총 픽셀 수_9600개
37          MOV    r2, #0             ; r2: 조건 만족하는 픽셀 수 count
38          MOV    r3, #0             ; r3: 현재 인덱스
39
40      loop
41          CMP    r3, r1
42          BEQ    done
43
44          LDRB   r4, [r0]
45          CMP    r4, #128
46          BLT    skip
47          ADD    r2, r2, #1
48
49      skip
50          ADD    r0, r0, #1
51          ADD    r3, r3, #1
52          B      loop
53
54      done
55          MOV    r0, r2
56          BX     lr
57
58      END
```

4. ARM 코드 최적화를 위한 C코드 및 ARM 코드

■ Subproject#1: 결과

Register	Value
Current	
R0	0x000012C3
R1	0x00002580
R2	0x000012C3
R3	0x00002580
R4	0x00000020
R5	0x00000012
R6	0x00000000
R7	0x00000000
R8	0x00000000
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13 (SP)	0x00000000
R14 (LR)	0x00000008
R15 (PC)	0x0000007C
CPSR	0x600000D3
SPSR	0x00000000

R0 register 값 = red 값이 128 이상인 pixel의 수

: $0x12C3 = 4803$

3에서 계산한 값과 같음을 알 수 있다.

4. ARM 코드 최적화를 위한 C코드 및 ARM 코드

■ Subproject#1: 결과 및 설명 (함수 수행 전 후의 memory map 정보)

Memory 1	
Address: 0x41000000	
0x41000000:	00 00
0x41000001A:	00 00
0x410000034:	00 00
0x41000004E:	00 00
0x410000068:	00 00
0x410000082:	00 00
0x41000009C:	00 00
0x4100000B6:	00 00
0x4100000D0:	00 00
0x4100000EA:	00 00
0x410000104:	00 00
0x41000011E:	00 00
0x410000138:	00 00
0x410000152:	00 00
0x41000016C:	00 00
0x410000186:	00 00
0x4100001A0:	00 00
0x4100001BA:	00 00
0x4100001D4:	00 00
0x4100001EE:	00 00
0x410000208:	00 00
0x410000222:	00 00
0x41000023C:	00 00
0x410000256:	00 00
0x410000270:	00 00
0x41000028A:	00 00
0x4100002A4:	00 00
0x4100002BE:	00 00
0x4100002D8:	00 00
0x4100002F2:	00 00
0x41000030C:	00 00
0x410000326:	00 00
0x410000340:	00 00
0x41000035A:	00 00
0x410000374:	00 00
0x41000038E:	00 00
0x4100003A8:	00 00
0x4100003C2:	00 00
0x4100003DC:	00 00

Memory 1	
Address: 0x41000000	
0x41000000:	78 77 92 21 77 67 FF F3 DD 82 67 80 A9 32 C2 F9 B5 33 D1 EB 89 DC 4D CD 6C F0
0x41000001A:	74 2C CD AE B4 DB 88 B6 B4 C6 AD 31 DA 44 46 EF CF 3E 4D 06 26 79 D8 F0 B1 31
0x410000034:	0D B3 CC 5F A3 E7 2B 86 CD 05 DE B8 B1 73 DC 44 6D D1 CD 45 6C 7B BC B1 13 D6
0x41000004E:	9D A2 B3 D3 6D 03 5A 35 FA 5C DF D7 55 4D 6A FA 74 C7 C6 69 87 3D D5 88 DB 2C
0x410000068:	AE F4 C6 B9 F5 6F 56 96 86 A3 7B E1 BE F2 31 9F B3 F7 E7 7D FA F5 EB CF DF 36
0x410000082:	BA B7 5C AD 68 99 2C F7 99 75 FA BF 87 E5 06 8E B1 2D 46 C4 87 38 3F F0 0F C5
0x41000009C:	FE BB FB 38 1A 3F 8A D0 FB E1 6B 51 91 FB 35 59 35 61 A5 AF 1A CF E7 88 BC D8
0x4100000B6:	F3 37 F1 98 DD C7 A7 97 0F 1C 8C D0 43 CB 5D 3C A3 DC F8 EA 78 3B 43 DD E7 79
0x4100000D0:	F6 5D 6A 66 DA A3 B6 8D AF D6 73 2D A5 3F 75 7E B9 BD D9 7A 2F E6 AC 5E CB 9F
0x4100000EA:	96 B7 AD 5B C6 50 D6 EE F0 BE 1F 74 96 58 4D 9E B5 0B A2 EC 7D 98 EB 1C D7 74
0x410000104:	05 16 53 EB 73 F6 0C 11 E1 D6 FB 11 2E D5 97 19 29 BE 74 36 10 5B CE 19 1D 6C
0x41000011E:	97 4D FE F6 24 A5 B1 13 23 7B B9 71 F3 99 9B 62 D7 F6 FE CB 7D C2 F4 AE EE 0F
0x410000138:	C1 7D 6D 1B 4D 9F BD 60 2C 37 03 25 3E DB FC B9 4B ED 14 08 F6 CD AD 9D 7C DB
0x410000152:	DA FA 2A 3D E3 D9 FC 65 B8 6B FB FB ED 9F 81 C8 4B C5 E1 DB CE ED CA 6A 40 E7
0x41000016C:	C4 3D D1 E2 27 3C 95 99 79 6D 06 6D E3 86 3A CE C1 E8 96 7B 8E D8 9B 9E 9E 9D
0x410000186:	6A D8 74 95 68 FC D9 E5 7E 60 7A 74 3F 22 95 5F F5 9C 7C 2F BD 4E 3E 75 05 98
0x4100001A0:	0E 27 98 E8 B0 D9 A9 AB 21 88 BC 21 F9 F9 D9 6D A6 B6 D9 4D D8 3C B6 BD B3 C7
0x4100001BA:	39 97 BF 6A 76 8E D5 10 D8 36 EB F4 3A 83 D2 89 33 31 AE 74 87 6E 7D 3F E7 2C
0x4100001D4:	52 E4 5C 67 8D B5 F3 B6 13 F1 87 8F 1C 4E 7B 6B E8 0B D9 DC 73 6F FE 9F 9F DF
0x4100001EE:	3A D3 16 C5 91 EB 5A 38 8D FC D0 38 7D 60 9E B6 1F 3E EB DD 78 3B 52 D8 1C 31
0x410000208:	74 B7 AD 50 20 1E 65 1C 82 CA 25 AC 3A 1C 99 74 8F 48 8F 58 80 12 DB FB C1 A2
0x410000222:	F9 7F 9A 22 0C C9 13 69 BE 31 6C 31 6B 04 11 9D FB 75 CD F4 4E EA B3 49 5A 00
0x41000023C:	F7 4A FE EF 39 A1 8A 32 40 EB E1 E4 45 BA B3 10 F7 25 F0 32 2D 05 27 BB 0B 08
0x410000256:	56 35 31 7F 77 9F 0D F0 0E 0F 3C 4F 38 D3 FB 36 71 70 78 29 01 04 44 D0 FD 8B
0x410000270:	73 88 0B 62 1B 53 A5 F3 15 E7 BE 09 4D DB BD 1A B6 06 C7 E2 C7 38 C7 D7 89 E2
0x41000028A:	9C F6 0C B9 B8 01 20 6F 86 CB C0 E4 08 4A B8 57 1B F3 F6 BF E0 4E 6C 13 67 12
0x4100002A4:	71 BC F3 EB 77 C2 DE 9C D9 C1 49 B1 D4 FC 66 35 1D 58 9D 88 4F 37 6A 2F 83 B8
0x4100002BE:	3C 6B 12 C8 E6 1C 7E 5C DC 5A 1D 10 AA 49 D0 C2 D9 CA 44 99 88 B3 9A 7B F1 6A
0x4100002D8:	BC AF 97 74 AD 70 9F 41 00 82 42 78 84 19 E6 53 9C CF B7 C5 3C D7 FB DF 7A 80
0x4100002F2:	E4 D6 1B 95 51 00 6B B5 FF 49 21 00 94 5A 70 96 C1 A2 0F 4C EC 72 DC 00 6B BF
0x41000030C:	B8 BE 37 C9 8B 00 71 E3 D0 5D 0E 7F EC C4 B5 67 31 D7 20 94 86 5E 68 5C B3 BB
0x410000326:	19 7A 8A 6C E5 27 9F FC 49 78 6C E2 E0 DC A2 BB CC 3C F3 F7 40 E9 F2 00 42 D6
0x410000340:	D7 79 4E 36 DD 69 24 DB EE 28 8F 57 9F 94 8A 85 7B 27 B8 CD 4C 67 67 DF 8D 23
0x41000035A:	AC 00 B0 E2 35 F9 AE 09 27 00 CC CF D2 4A 80 38 21 AA 08 AB D3 4A 32 D6 4D EE
0x410000374:	92 ED 93 B1 2D 90 4F D8 5A 12 30 AF 18 76 0D C0 FD 81 C6 F7 92 4C 00 04 C5 C3
0x41000038E:	A7 E9 FF FF 72 B9 9C F0 D9 4B F1 BE BF BE E7 FC B6 A3 A7 0C D5 12 4B AC F6 B1
0x4100003A8:	34 9D 67 A4 B0 A4 A8 C3 90 C6 EF 5A 8A C3 AA 0B 37 DA 1C 34 D6 C7 1E 79 12 7A
0x4100003C2:	04 DA 88 DD 0A 64 7D A1 AE 98 98 08 2B 84 0A A4 B8 40 59 07 18 50 33 46 AA 67
0x4100003DC:	80 44 12 5D A1 77 4A B1 10 8D B1 61 04 AA 91 D9 63 9D 43 49 4F E9 A6 25 54 B7

4. ARM 코드 최적화를 위한 C코드 및 ARM 코드

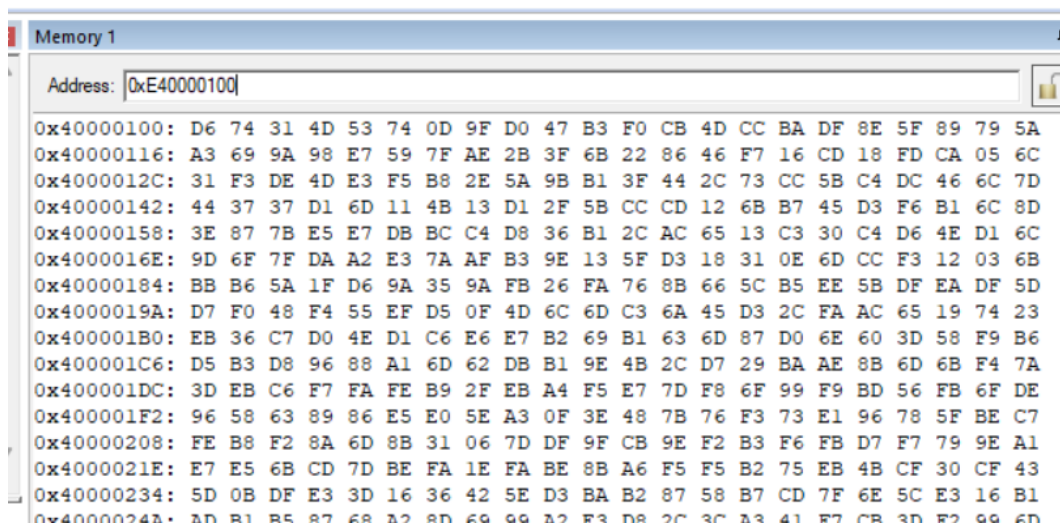
■ Subproject#2: Assem.s

```
1      AREA Reset, CODE, READONLY
2      EXPORT start
3
4      start
5          BL relocate
6          BL re_rgb_negative
7
8      relocate
9          LDR    r0, =0x40000036
10         LDR    r1, =0x41000000
11         LDR    r2, =0x41002580
12         LDR    r3, =0x41004B00
13         MOV    r4, #9600
14
15     loop_relocate
16         LDRB    r5, [r0], #1
17         STRB    r5, [r1], #1
18
19         LDRB    r5, [r0], #1
20         STRB    r5, [r2], #1
21
22         LDRB    r5, [r0], #1
23         STRB    r5, [r3], #1
24
25         ADD    r0, r0, #1
26         SUBS    r4, r4, #1
27         BNE    loop_relocate
28         BX     lr
29
```

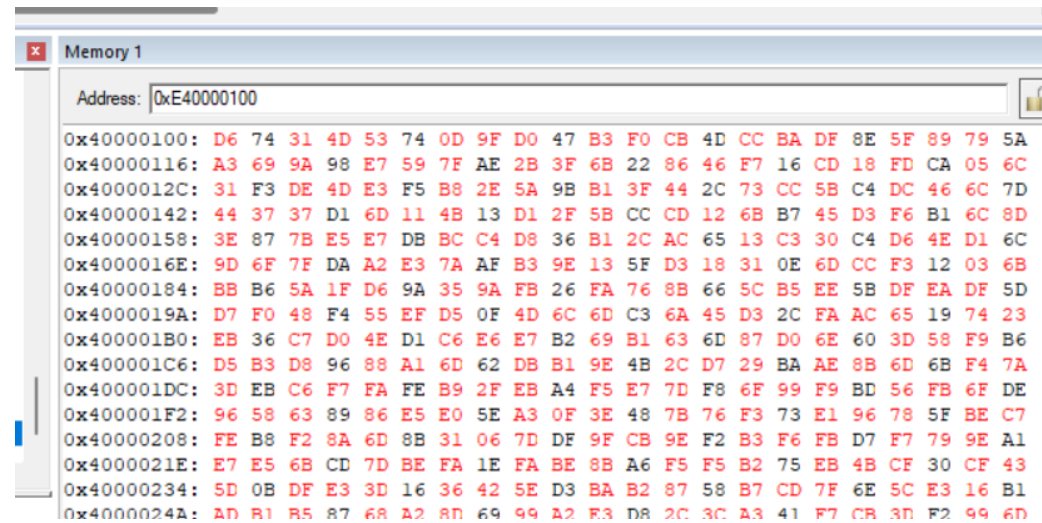
```
30     re_rgb_negative
31         LDR    r0, =0x41000000
32         LDR    r1, =0x41002580
33         LDR    r2, =0x41004B00
34         LDR    r3, =0x20000000
35         MOV    r4, #9600
36
37     loop_negative
38         LDRB    r5, [r0], #1
39         RSBS    r5, r5, #255
40         STRB    r5, [r3], #1
41
42         LDRB    r6, [r1], #1
43         RSBS    r6, r6, #255
44         STRB    r6, [r3], #1
45
46         LDRB    r7, [r2], #1
47         RSBS    r7, r7, #255
48         STRB    r7, [r3], #1
49
50         SUBS    r4, r4, #1
51         BNE    loop_negative
52
53         BX     lr
54
55     END
```


4. ARM 코드 최적화를 위한 C코드 및 ARM 코드

▪ Subproject#2: 결과 및 설명 (함수 수행 전 후의 memory map 정보)



```
Memory 1
Address: 0xE40000100
0xE40000100: D6 74 31 4D 53 74 0D 9F D0 47 B3 F0 CB 4D CC BA DF 8E 5F 89 79 5A
0xE40000116: A3 69 9A 98 E7 59 7F AE 2B 3F 6B 22 86 46 F7 16 CD 18 FD CA 05 6C
0xE4000012C: 31 F3 DE 4D E3 F5 B8 2E 5A 9B B1 3F 44 2C 73 CC 5B C4 DC 46 6C 7D
0xE40000142: 44 37 37 D1 6D 11 4B 13 D1 2F 5B CC CD 12 6B B7 45 D3 F6 B1 6C 8D
0xE40000158: 3E 87 7B E5 E7 DB BC C4 D8 36 B1 2C AC 65 13 C3 30 C4 D6 4E D1 6C
0xE4000016E: 9D 6F 7F DA A2 E3 7A AF B3 9E 13 5F D3 18 31 0E 6D CC F3 12 03 6B
0xE40000184: BB B6 5A 1F D6 9A 35 9A FB 26 FA 76 8B 66 5C B5 EE 5B DF EA DF 5D
0xE4000019A: D7 F0 48 F4 55 EF D5 0F 4D 6C 6D C3 6A 45 D3 2C FA AC 65 19 74 23
0xE400001B0: EB 36 C7 D0 4E D1 C6 E6 E7 B2 69 B1 63 6D 87 D0 6E 60 3D 58 F9 B6
0xE400001C6: D5 B3 D8 96 88 A1 6D 62 DB B1 9E 4B 2C D7 29 BA AE 8B 6D 6B F4 7A
0xE400001DC: 3D EB C6 F7 FA FE B9 2F EB A4 F5 E7 7D F8 6F 99 F9 BD 56 FB 6F DE
0xE400001F2: 96 58 63 89 86 E5 E0 5E A3 0F 3E 48 7B 76 F3 73 E1 96 78 5F BE C7
0xE40000208: FE B8 F2 8A 6D 8B 31 06 7D DF 9F CB 9E F2 B3 F6 FB D7 F7 79 9E A1
0xE4000021E: E7 E5 6B CD 7D BE FA 1E FA BE 8B A6 F5 F5 B2 75 EB 4B CF 30 CF 43
0xE40000234: 5D 0B DF E3 3D 16 36 42 5E D3 BA B2 87 58 B7 CD 7F 6E 5C E3 16 B1
0xE4000024A: AD B1 B5 87 68 A2 8D 69 99 A2 F3 D8 2C 3C A3 41 F7 CB 3D F2 99 6D
```



```
Memory 1
Address: 0xE40000100
0xE40000100: D6 74 31 4D 53 74 0D 9F D0 47 B3 F0 CB 4D CC BA DF 8E 5F 89 79 5A
0xE40000116: A3 69 9A 98 E7 59 7F AE 2B 3F 6B 22 86 46 F7 16 CD 18 FD CA 05 6C
0xE4000012C: 31 F3 DE 4D E3 F5 B8 2E 5A 9B B1 3F 44 2C 73 CC 5B C4 DC 46 6C 7D
0xE40000142: 44 37 37 D1 6D 11 4B 13 D1 2F 5B CC CD 12 6B B7 45 D3 F6 B1 6C 8D
0xE40000158: 3E 87 7B E5 E7 DB BC C4 D8 36 B1 2C AC 65 13 C3 30 C4 D6 4E D1 6C
0xE4000016E: 9D 6F 7F DA A2 E3 7A AF B3 9E 13 5F D3 18 31 0E 6D CC F3 12 03 6B
0xE40000184: BB B6 5A 1F D6 9A 35 9A FB 26 FA 76 8B 66 5C B5 EE 5B DF EA DF 5D
0xE4000019A: D7 F0 48 F4 55 EF D5 0F 4D 6C 6D C3 6A 45 D3 2C FA AC 65 19 74 23
0xE400001B0: EB 36 C7 D0 4E D1 C6 E6 E7 B2 69 B1 63 6D 87 D0 6E 60 3D 58 F9 B6
0xE400001C6: D5 B3 D8 96 88 A1 6D 62 DB B1 9E 4B 2C D7 29 BA AE 8B 6D 6B F4 7A
0xE400001DC: 3D EB C6 F7 FA FE B9 2F EB A4 F5 E7 7D F8 6F 99 F9 BD 56 FB 6F DE
0xE400001F2: 96 58 63 89 86 E5 E0 5E A3 0F 3E 48 7B 76 F3 73 E1 96 78 5F BE C7
0xE40000208: FE B8 F2 8A 6D 8B 31 06 7D DF 9F CB 9E F2 B3 F6 FB D7 F7 79 9E A1
0xE4000021E: E7 E5 6B CD 7D BE FA 1E FA BE 8B A6 F5 F5 B2 75 EB 4B CF 30 CF 43
0xE40000234: 5D 0B DF E3 3D 16 36 42 5E D3 BA B2 87 58 B7 CD 7F 6E 5C E3 16 B1
0xE4000024A: AD B1 B5 87 68 A2 8D 69 99 A2 F3 D8 2C 3C A3 41 F7 CB 3D F2 99 6D
```

- 원래 방식: RGBA 이미지 메모리를 연속적으로 접근하면서 R->G->B->A 를 한 번에 처리하는 (포인터 하나, 순차 접근) 방식
- Relocation 방식: 한 번 RGBA를 읽고, R/G/B를 따로 저장하는 (Memory copy) 방식
>> 그 후 R, G, B 영역을 각각 독립적으로 접근하여 후처리

4. ARM 코드 최적화를 위한 C코드 및 ARM 코드

■ Subproject#2: 결과 및 설명 (함수 수행 전 후의 memory map 정보)

Register	Value
Current	
R0	0x41002580
R1	0x41004B00
R2	0x41007080
R3	0x20007080
R4	0x00000000
R5	0x0000000F
R6	0x0000004B
R7	0x000000ED
R8	0x00000000
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13 (SP)	0x00000000
R14 (LR)	0x00000008
R15 (PC)	0x00000084
CPSR	0x60000003
SPSR	0x00000000
User/System	
Fast Interrupt	
Interrupt	
Supervisor	
Abort	
Undefined	
Internal	
PC \$	0x00000084
Mode	Supervisor
States	403223
Sec	0,00672038

Address: 0x20000000	
0x20000000:	87 A1 A3 88 8C DC 6D 13 E4 DE F7 4D 88 70 A7 98 10 4A 00 84 20 0C C3 4C 22 62
0x2000001A:	92 7D FD BF 98 C0 08 7F 4C E3 56 D5 6D CD B0 61 3D 3C 3C 06 0F 0E 4A 82 E8 CC
0x20000034:	3A 2D 2E 49 92 14 CA 79 76 22 2F 23 B0 2E B2 CB A2 32 45 3A 93 4E 49 0F 4A 92
0x2000004E:	8B CE 39 D3 2E 38 32 CD 38 51 4E 49 4B 32 E7 24 B9 4B 77 34 EB 49 CE CB 4B 43
0x20000068:	B8 39 2B 93 52 B4 3B CE 0B 44 25 AA 10 BB E8 A2 B9 CB 01 10 E8 B2 30 74 92 C1
0x20000082:	85 1A B2 EC 14 F9 80 92 D9 45 99 86 1A 68 27 5D 0A 0F 04 FC 4E 39 29 CE B2 AC
0x2000009C:	F2 60 2F 4C 0F 34 33 45 20 A0 76 86 5C 96 65 18 A6 80 D4 C0 94 79 B9 08 32 E7
0x200000B6:	02 FA 93 CE 21 B2 1C 47 D1 A5 4E C0 BB 8C 33 A4 23 B9 93 BB C8 C8 92 EE B4 2E
0x200000D0:	D0 A4 32 ED 94 BA 2C 09 93 72 C1 84 1A 18 43 3B 27 4E D3 53 EC 3C CF 29 B1 2E
0x200000EA:	62 90 80 5D 1C 85 4C 61 EC 2C E7 CE 92 33 0C FC 94 44 A5 E0 29 CA 65 04 05 89
0x20000104:	74 A3 4A 11 20 15 20 28 0F B7 AA 10 2A B2 93 92 95 BA 2C 05 53 9A 8B DC 14 38
0x2000011E:	2F B1 39 19 18 96 4E 9C 78 2F 91 C2 A7 06 2A 4C 27 77 5E 92 24 4E 61 D3 28 D6
0x20000138:	51 74 92 0B 85 C2 39 08 05 46 D0 14 0A 18 82 90 66 06 A9 04 90 69 A7 9C 79 1A
0x20000152:	1F 5C F0 C1 84 89 0C 1E 69 87 41 38 01 0D 75 92 CE F9 82 60 34 61 4C 09 04 08
0x2000016C:	86 61 18 1A 94 82 41 05 05 41 74 0A 0A 4D 14 B4 30 30 BC A2 20 1C C2 C9 BD A1
0x20000186:	45 4D 78 48 32 80 A3 1C E9 52 4E 4A 97 5D 72 66 5D 1C D3 C3 5C 08 34 C2 66 92
0x200001A0:	44 8A 0E F8 05 4E 08 40 4A 59 78 49 14 1A FE 93 F9 92 77 71 C9 8D 4E 91 93 D2
0x200001BA:	DC C8 B9 2C 71 3B 5B 8A 78 77 E7 C7 E3 88 C0 CB 0E 0F EB B0 F0 4E C0 3A 9E C3
0x200001D4:	01 9E E8 44 CE E1 04 E7 B1 C7 13 F0 E5 61 1D C0 3A 16 75 8E C7 2F 72 25 04 09
0x200001EE:	D6 1E E5 A2 94 00 8A AE 84 90 6E 84 76 04 CA B0 CA 5D A6 A6 30 EE CA 49 67 9E
0x20000208:	06 A4 5A 2E 46 50 AC B3 E5 68 96 30 28 C9 18 C1 61 77 1A 45 43 0E 63 27 69 D1
0x20000222:	0C 2A 08 C8 8E 21 0E 8D 86 67 38 67 22 7A 0C 38 8E 80 58 38 38 68 E0 81 F0 B0
0x2000023C:	F0 E3 44 87 73 EE A2 2F 8A 2E BC B3 D4 34 C1 A4 A2 50 85 C3 4C 49 5C 76 DE 23
0x20000256:	44 94 07 86 19 15 5C 74 87 2F 41 C4 10 B0 BC 3C 46 22 27 9D 18 37 A6 86 E0 13
0x20000270:	09 24 55 A2 19 BA 95 72 D1 99 6A 52 25 DE 5D 5C E4 05 49 C9 65 72 51 20 50 49

- 레지스터에 저장된 주소 값과 메모리 0x20000000에 저장된 결과를 통해 3과 동일한 결과를 얻음

4. ARM 코드 최적화를 위한 C코드 및 ARM 코드

■ Subproject#3: Assem.s

```
1      AREA Reset, CODE, READONLY
2      EXPORT start
3
4      start
5          BL relocate
6          BL re_grayscale
7
8      relocate
9          LDR    r0, =0x40000036
10         LDR    r1, =0x41000000
11         LDR    r2, =0x41002580
12         LDR    r3, =0x41004B00
13         MOV    r4, #9600
14
15      loop_relocate
16          LDRB   r5, [r0], #1
17          STRB   r5, [r1], #1
18
19          LDRB   r5, [r0], #1
20          STRB   r5, [r2], #1
21
22          LDRB   r5, [r0], #1
23          STRB   r5, [r3], #1
24
25          ADD    r0, r0, #1      ; A 무시
26          SUBS   r4, r4, #1
27          BNE    loop_relocate
28          BX     lr
29
```

```
30      re_grayscale
31          LDR    r0, =0x41000000      ; R
32          LDR    r1, =0x41002580      ; G
33          LDR    r2, =0x41004B00      ; B
34          LDR    r3, =0x20000000      ; Grayscale 저장 주소
35          MOV    r4, #9600            ; 픽셀 수
36
37      loop_gray
38          LDRB   r5, [r0], #1          ; R
39          LDRB   r6, [r1], #1          ; G
40          LDRB   r7, [r2], #1          ; B
41
42          ; grayscale = (2R + 6G + B) >> 3
43          ADD    r8, r5, r6, LSL #1
44          ADD    r8, r8, r8, LSL #1
45          ADD    r8, r8, r7
46
47          STRH   r8, [r3], #2          ; 1바이트로 저장
48
49          SUBS   r4, r4, #1
50          BNE    loop_gray
51
52      END
```

4. ARM 코드 최적화를 위한 C코드 및 ARM 코드

■ Subproject#3: 결과 및 설명 (함수 수행 전 후의 memory map 정보)

Address: 0x40000000
0x40000000: 89 50 4E 47 0D 0A 1A 0A 00 00 00 0D 49 48 44 52 00 00 03 C0
0x40000004: 00 00 02 80 08 06 00 00 00 53 F9 11 43 00 00 01 73 52 47
0x40000008: 42 00 AE CE 1C E9 00 00 20 00 49 44 41 54 78 5E 5C BD 77 73
0x4000000C: 23 59 92 EC 1B 29 21 08 B2 44 77 8F 58 71 67 EF B5 77 FF 7B
0x40000010: DF FF F3 3C B3 B5 D0 9D 6D 59 82 02 40 AA 67 3F F7 08 80 B3
0x40000014: 1C EB A9 2A 92 00 32 4F 9E 13 C2 C3 C3 A3 F9 F0 F1 B8 B5 7D
0x40000018: 17 73 33 C5 D2 75 D1 B6 6D 6C EB 35 86 B5 89 DD D0 C7 DC 4F
0x4000001C: D1 F6 4D 34 5D 1B CD BA C5 D6 6C B1 B6 5B F0 B5 6D 5B 74 31
0x40000020: C6 DA 2C D1 C7 16 CD 32 C7 D2 AE B1 B6 5D B4 CD 18 6B DB 46
0x40000024: B4 E7 89 CB 14 D1 B6 31 34 6D B4 BC 47 D7 C6 D4 6C 31 AD 4B
0x40000028: C4 14 31 F4 BB D8 DA 55 EF D7 44 17 5D D3 46 34 FE 37 EF 17
0x4000002C: 4D A3 CF 8B 6D 08 3E 7A E5 9F 4D 13 EB BA 06 7F 6D DB 26 BA
0x40000030: 66 8B 79 E5 97 9A D8 A2 F5 F5 F0 FB 03 BF B1 C6 D6 74 31 4D
0x40000034: 53 74 0D 9F D0 47 B3 F0 CB 4D CC BA DF 8E 5F 89 79 5A A3 69
0x40000038: 9A 98 E7 59 7F AE 2B 3F 6B 22 86 46 F7 16 CD 18 FD CA 05 6C
0x4000003C: 31 F3 DE 4D E3 F5 B9 2E 5A 9B B1 3F 44 2C 73 CC 5B C4 DC 46
0x40000040: 6C 7D 44 37 37 D1 6D 11 4B 13 D1 2F 5B CC CD 12 6B B7 45 D3
0x40000044: F6 B1 6C 8D 3E 87 7B E5 E7 DB BC C4 D8 36 B1 2C AC 65 13 C3
0x40000048: 30 C4 D6 4E D1 6C 9D 6F 7F DA A2 E3 7A AF B3 9E 13 5F D3 18
0x4000004C: 31 0E 6D CC F3 12 03 6B BB B6 5A 1F D6 9A 35 9A FB 26 FA 76
0x40000050: 8B 66 5C B5 EE 5B DF EA DF 5D D7 F0 48 F4 55 EF D5 0F 4D 6C
0x40000054: 6D C3 6A 45 D3 2C FA AC 65 19 74 23 EB 36 C7 D0 4E D1 C6 E6
0x40000058: E7 B2 69 B1 63 6D 87 D0 6E 60 3D 58 F9 B6 D5 B3 D8 96 88 A1

Register	Value
Current	
R0	0x40000038
R1	0x41000001
R2	0x41002581
R3	0x41004B00
R4	0x00002580
R5	0x0000005E
R6	0x00000000
R7	0x00000000
R8	0x00000000
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13 (SP)	0x00000000
R14 (LR)	0x00000004
R15 (PC)	0x0000002C
CPSR	0x00000003
SPSR	0x00000000

- Grayscale 값은 $(3 \times R + 6 \times G + B)$ 를 8로 나눈 값으로 산출하며, 계산된 값을 16bit 단위로 지정된 메모리 주소에 저장
- 이 과정을 9600번 반복해 RGB 데이터를 Grayscale 이미지로 변환

Address: 0x20000000
0x20000000: 6B 12 00 46 01 01 04 7C
0x20000008: 21 3A 57 74 24 5C 8C 5A
0x20000010: 22 7B 64 BB EF C8 6D 75
0x20000018: 7F 83 49 6A CE D3 45 A3
0x20000020: AB 9D D1 A1 74 A1 A5 67
0x20000028: 83 71 C0 89 79 B7 72 74
0x20000030: 8F 6F 6C CA 99 8C 9A 8E
0x20000038: 3C 99 78 80 78 B7 F2 61
0x20000040: 83 40 8B 8C 9F 7E BF 6C

0x2002570
0x200257F4: 81 92 95 49
0x200257F8: 2E 5F 8E 94
0x200257FC: B3 1C 4A 6A
0x20025800: 00 00 00 00
0x20025804: 00 00 00 00
0x20025808: 00 00 00 00
0x2002580C: 00 00 00 00
0x20025810: 00 00 00 00
0x20025814: 00 00 00 00

5. 최적의 ARM 코드 탐색

1. ARM 코드 최적화를 위한 방향 설정

- 실험 목적 : 플래너 방식으로 메모리에 재배치된 이미지 hex 파일을 바탕으로 이미지 변환 함수를 시뮬레이션
- ARM 코드 최적화 목표 : 코드 크기 최소화 & 실행 속도 향상
 - > 루프 내부 명령어의 단순화 & 메모리 접근 효율화 초점

2. 최적화 전략

① 복합 명령어 적극 활용

- ARM 명령어 중 일부는 조건 플래그(NZCV)를 자동 설정하는 기능 존재
 - > CMP R1, R2는 비교만 수행하고 조건 플래그를 설정
 - > SUBS R1, R2, R3는 뺄셈 + 조건 플래그 설정을 동시 수행
- 단순한 비교 후 분기하는 구조보다 "연산과 조건 비교를 동시에 수행하는 명령어 사용"
 - > 명령어 수를 줄이고 코드 간결성 확보 가능
- 루프 탈출 조건 등을 단일 명령어로 처리해 **루프 내 실행 효율 향상** 유도

5. 최적의 ARM 코드 탐색

② 메모리 접근 최소화

- LDR, STR 같은 메모리 접근 명령어는 CPU 레지스터 간 연산보다 훨씬 느림
- 따라서 동일한 값을 반복적으로 참조해야 할 경우
 - > 레지스터에 저장해두고 활용
 - > 불필요한 메모리 재접근을 피하는 것이 중요
- 메모리 접근 최소화를 통해 전반적인 명령어 실행 속도 개선 기대 가능

③ 루프 언롤링 (Loop Unrolling)

- 루프 언롤링: 반복문 안의 코드를 복제하여 한 번에 여러 작업 수행
 - > 루프 조건 비교 및 분기 명령어의 실행 횟수 자체를 줄이는 방식
- 이번 실험: 루프마다 동일한 이미지 변환 연산을 반복 수행 (연산 자체는 복잡하지 않아 효과적)
- 효과: 분기 명령어의 실행 횟수 감소 & 전체 루프 반복 횟수 감소 (실행 시간 단축)

5. 최적의 ARM 코드 탐색

3. Function#1 – Red pixel count [기존 코드]

```
1      AREA Reset, CODE, READONLY
2      EXPORT start
3
4      start
5          BL relocate
6          BL Red_Pixel_Count_Re
7
8      relocate
9          LDR    r0, =0x40000036    ; r0 = RGBA 데이터 시작 주소
10         LDR    r1, =0x41000000    ; r1 = R값 저장할 목적지 주소
11         LDR    r2, =0x41002580    ; r2 = G값 저장할 목적지 주소
12         LDR    r3, =0x41004B00    ; r3 = B값 저장할 목적지 주소
13         MOV    r4, #9600          ; r4 = 픽셀 수
14
15      loop_relocate
16          LDRB   r5, [r0], #1        ;
17          STRB   r5, [r1], #1        ; R 저장
18
19          LDRB   r5, [r0], #1        ;
20          STRB   r5, [r2], #1        ; G 저장
21
22          LDRB   r5, [r0], #1        ;
23          STRB   r5, [r3], #1        ; B 저장
24
25          ADD    r0, r0, #1          ; A 무시
26
27          SUBS   r4, r4, #1
28
29          BNE    loop_relocate
30          BX     lr ; 재정렬 끝
```

```
34      Red_Pixel_Count_Re
35          LDR    r0, =0x41000000
36          MOV    r1, #9600          ; r1: 총 픽셀 수_9600개
37          MOV    r2, #0             ; r2: 조건 만족하는 픽셀 수 count
38          MOV    r3, #0             ; r3: 현재 인덱스
39
40      loop
41          CMP    r3, r1
42          BEQ    done
43
44          LDRB   r4, [r0]
45          CMP    r4, #128
46          BLT    skip
47          ADD    r2, r2, #1
48
49      skip
50          ADD    r0, r0, #1
51          ADD    r3, r3, #1
52          B      loop
53
54      done
55          MOV    r0, r2
56          BX     lr
57
58      END
```

5. 최적의 ARM 코드 탐색

3. Function#1 – Red pixel count [최적화 후 코드]

```
1      AREA    RelocateAndCount, CODE, READONLY
2      EXPORT  relocate_and_count_red
3      ENTRY
4
5  relocate_and_count_red
6      PUSH    {R4-R9, LR}
7
8      MOV     R1, #0
9      LDR     R2, =0x40000036      ; RGBA ?? ??
10     LDR     R3, =0x50000000      ; R ???
11     LDR     R4, =0x50002580      ; G ???
12     LDR     R5, =0x50004B00      ; B ???
13
14 ; === [1??] RGBA -> RRR... GGG... BBB... ?? ?? ===
15 relocate_loop
16     CMP     R1, #9600
17     BGE     count_start
18
19     LDRB    R6, [R2]              ; R
20     LDRB    R7, [R2, #1]         ; G
21     LDRB    R8, [R2, #2]         ; B
22
23     STRB    R6, [R3], #1         ; R ??
24     STRB    R7, [R4], #1         ; G ??
25     STRB    R8, [R5], #1         ; B ??
26
27     ADD     R2, R2, #4
28     ADD     R1, R1, #1
29     B       relocate_loop
```

```
32 count_start
33     MOV     R0, #0
34     MOV     R1, #1600
35     LDR     R3, =0x50000000
36
37 count_loop
38     LDRB    R6, [R3], #1
39     CMP     R6, #128
40     ADDGE   R0, R0, #1
41
42     LDRB    R6, [R3], #1
43     CMP     R6, #128
44     ADDGE   R0, R0, #1
45
46     LDRB    R6, [R3], #1
47     CMP     R6, #128
48     ADDGE   R0, R0, #1
49
50     LDRB    R6, [R3], #1
51     CMP     R6, #128
52     ADDGE   R0, R0, #1
53
54     LDRB    R6, [R3], #1
55     CMP     R6, #128
56     ADDGE   R0, R0, #1
57
58     LDRB    R6, [R3], #1
59     CMP     R6, #128
60     ADDGE   R0, R0, #1
61
62
63     SUBS    R1, R1, #1
64     BNE     count_loop
```

```
66 done
67     POP     {R4-R9, PC}
68
69     END
70
```

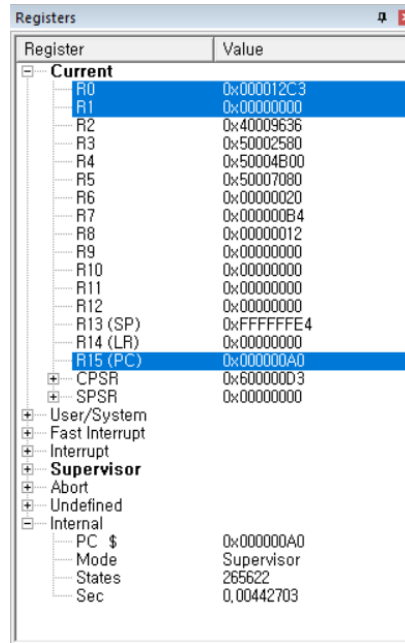

5. 최적의 ARM 코드 탐색

3. Function#1 – Red pixel count [최적화 후 코드]

- relocate_loop까지 실행해서 0x50000000에 색상 채널들을 재배치 후 128비트 이상의 레드 픽셀 수를 1바이트씩 접근하여 계산
- 128비트 레드 픽셀을 세는 과정에서 명령어 수를 줄이기 위해 ADDGE를 활용
- 최적화 이후 코드에서는 이 과정을 skip 하는 함수를 새로 만들어 분기하지 않고 ADDGE를 통해 한 번에 수행
- 수행 시간을 줄이기 위해서 루프 언롤링을 수행
- 4,6,8씩 루프 언롤링 횟수를 늘려가면서 실험 >> 루프 언롤링 횟수를 늘릴수록 시간이 줄긴 했지만 유지보수와 코드의 가독성 등을 고려하여 언롤링 횟수는 6으로 지정

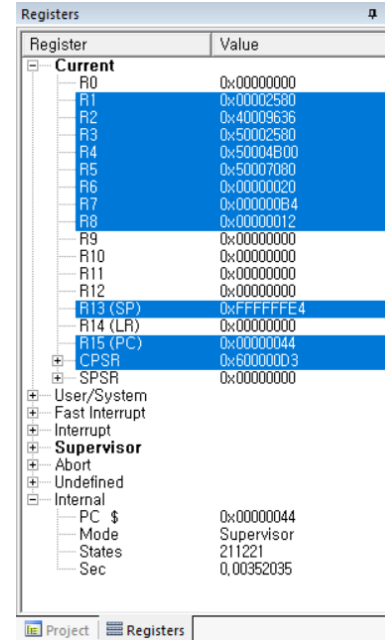
5. 최적의 ARM 코드 탐색

3. Function#1 – Red pixel count [최적화 후 코드]



Register	Value
Current	
R0	0x000012C3
R1	0x00000000
R2	0x40009636
R3	0x50002580
R4	0x50004B00
R5	0x50007080
R6	0x00000020
R7	0x000000B4
R8	0x00000012
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13 (SP)	0xFFFFFE4
R14 (LR)	0x00000000
R15 (PC)	0x000000A0
CPSR	0x60000003
SPSR	0x00000000
User/System	
Fast Interrupt	
Interrupt	
Supervisor	
Abort	
Undefined	
Internal	
PC \$	0x000000A0
Mode	Supervisor
States	265622
Sec	0.00442703

[최적화 코드 전체 수행시간]



Register	Value
Current	
R0	0x00000000
R1	0x00002580
R2	0x40009636
R3	0x50002580
R4	0x50004B00
R5	0x50007080
R6	0x00000020
R7	0x000000B4
R8	0x00000012
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13 (SP)	0xFFFFFE4
R14 (LR)	0x00000000
R15 (PC)	0x000000A4
CPSR	0x60000003
SPSR	0x00000000
User/System	
Fast Interrupt	
Interrupt	
Supervisor	
Abort	
Undefined	
Internal	
PC \$	0x000000A4
Mode	Supervisor
States	211221
Sec	0.00352035

[최적화 코드 전체 메모리 재배치 시간]

- 전체 수행시간 – 메모리 재배치 시간 = 이미지 변환 함수 수행시간 = 0.00090668sec
- 수행시간 단축은 기존의 약 2.1ms 보다 1.2ms 줄어든 수행시간을 얻었다.

5. 최적의 ARM 코드 탐색

4. Function#2 – Image negative [기존 코드]

```
1      AREA Reset, CODE, READONLY
2      EXPORT start
3
4      start
5          BL relocate
6          BL re_rgb_negative
7
8      relocate
9          LDR    r0, =0x40000036
10         LDR    r1, =0x41000000
11         LDR    r2, =0x41002580
12         LDR    r3, =0x41004B00
13         MOV    r4, #9600
14
15     loop_relocate
16         LDRB    r5, [r0], #1
17         STRB    r5, [r1], #1
18
19         LDRB    r5, [r0], #1
20         STRB    r5, [r2], #1
21
22         LDRB    r5, [r0], #1
23         STRB    r5, [r3], #1
24
25         ADD    r0, r0, #1
26         SUBS    r4, r4, #1
27         BNE    loop_relocate
28         BX     lr
```

```
30     re_rgb_negative
31         LDR    r0, =0x41000000
32         LDR    r1, =0x41002580
33         LDR    r2, =0x41004B00
34         LDR    r3, =0x20000000
35         MOV    r4, #9600
36
37     loop_negative
38         LDRB    r5, [r0], #1
39         RSBS    r5, r5, #255
40         STRB    r5, [r3], #1
41
42         LDRB    r6, [r1], #1
43         RSBS    r6, r6, #255
44         STRB    r6, [r3], #1
45
46         LDRB    r7, [r2], #1
47         RSBS    r7, r7, #255
48         STRB    r7, [r3], #1
49
50         SUBS    r4, r4, #1
51         BNE    loop_negative
52
53         BX     lr
54
55     END
```

5. 최적의 ARM 코드 탐색

4. Function#2 – Image negative [최적화 후 코드]

```
1      AREA Reset, CODE, READONLY
2      EXPORT start
3
4      start
5          BL relocate
6          BL re_rgb_negative
7
8      relocate
9          LDR    r0, =0x40000036
10         LDR    r1, =0x41000000
11         LDR    r2, =0x41002580
12         LDR    r3, =0x41004B00
13         MOV    r4, #9600
14
15     loop_relocate
16         LDRB   r5, [r0], #1
17         STRB   r5, [r1], #1
18
19         LDRB   r5, [r0], #1
20         STRB   r5, [r2], #1
21
22         LDRB   r5, [r0], #1
23         STRB   r5, [r3], #1
24
25         ADD    r0, r0, #1
26         SUBS   r4, r4, #1
27         BNE    loop_relocate
28         BX     lr
29
```

```
30     re_rgb_negative
31         LDR    r0, =0x41000000
32         LDR    r1, =0x41002580
33         LDR    r2, =0x41004B00
34         LDR    r3, =0x20000000
35         MOV    r4, #1600
36
37     loop_negative
38         LDRB   r5, [r0], #1
39         RSB    r5, r5, #255
40
41         LDRB   r6, [r1], #1
42         RSB    r6, r6, #255
43
44         LDRB   r7, [r2], #1
45         RSB    r7, r7, #255
46
47         STRB   r5, [r3], #1
48         STRB   r6, [r3], #1
49         STRB   r7, [r3], #1
50
51         STRB   r5, [r3], #1
52         STRB   r6, [r3], #1
53         STRB   r7, [r3], #1
54
```

```
54         LDRB   r5, [r0], #1
55         RSB    r5, r5, #255
56
57         LDRB   r6, [r1], #1
58         RSB    r6, r6, #255
59
60         LDRB   r7, [r2], #1
61         RSB    r7, r7, #255
62
63         STRB   r5, [r3], #1
64         STRB   r6, [r3], #1
65         STRB   r7, [r3], #1
66
67         LDRB   r5, [r0], #1
68         RSB    r5, r5, #255
69
70         LDRB   r6, [r1], #1
71         RSB    r6, r6, #255
72
73         LDRB   r7, [r2], #1
74         RSB    r7, r7, #255
75
76         STRB   r5, [r3], #1
77         STRB   r6, [r3], #1
78         STRB   r7, [r3], #1
79
80         STRB   r5, [r3], #1
81         STRB   r6, [r3], #1
82         STRB   r7, [r3], #1
83
84         STRB   r5, [r3], #1
85         STRB   r6, [r3], #1
86         STRB   r7, [r3], #1
87
```

5. 최적의 ARM 코드 탐색

4. Function#2 – Image negative [최적화 후 코드]

```
86      LDRB    r5, [r0], #1
87      RSB     r5, r5, #255
88
89
90      LDRB    r6, [r1], #1
91      RSB     r6, r6, #255
92
93
94      LDRB    r7, [r2], #1
95      RSB     r7, r7, #255
96
97
98      STRB     r5, [r3], #1
99      STRB     r6, [r3], #1
100     STRB     r7, [r3], #1
101
102     LDRB     r5, [r0], #1
103     RSB      r5, r5, #255
104
105
106     LDRB     r6, [r1], #1
107     RSB      r6, r6, #255
108
109
110     LDRB     r7, [r2], #1
111     RSB      r7, r7, #255
112
113
114     STRB     r5, [r3], #1
115     STRB     r6, [r3], #1
116     STRB     r7, [r3], #1
117
```

```
118     LDRB     r5, [r0], #1
119     RSB      r5, r5, #255
120
121
122     LDRB     r6, [r1], #1
123     RSB      r6, r6, #255
124
125
126     LDRB     r7, [r2], #1
127     RSB      r7, r7, #255
128
129
130     STRB     r5, [r3], #1
131     STRB     r6, [r3], #1
132     STRB     r7, [r3], #1
133
134     SUBS     r4, r4, #1
135     BNE      loop_negative
136
137     BX       lr
138
139     END
```

5. 최적의 ARM 코드 탐색

4. Function#2 – Image negative [최적화 후 코드]

```
86      LDRB    r5, [r0], #1
87      RSB     r5, r5, #255
88
89
90      LDRB    r6, [r1], #1
91      RSB     r6, r6, #255
92
93
94      LDRB    r7, [r2], #1
95      RSB     r7, r7, #255
96
97
98      STRB    r5, [r3], #1
99      STRB    r6, [r3], #1
100     STRB    r7, [r3], #1
101
102     LDRB    r5, [r0], #1
103     RSB     r5, r5, #255
104
105
106     LDRB    r6, [r1], #1
107     RSB     r6, r6, #255
108
109
110     LDRB    r7, [r2], #1
111     RSB     r7, r7, #255
112
113
114     STRB    r5, [r3], #1
115     STRB    r6, [r3], #1
116     STRB    r7, [r3], #1
117
```

```
118     LDRB    r5, [r0], #1
119     RSB     r5, r5, #255
120
121
122     LDRB    r6, [r1], #1
123     RSB     r6, r6, #255
124
125
126     LDRB    r7, [r2], #1
127     RSB     r7, r7, #255
128
129
130     STRB    r5, [r3], #1
131     STRB    r6, [r3], #1
132     STRB    r7, [r3], #1
133
134     SUBS    r4, r4, #1
135     BNE     loop_negative
136
137     BX      lr
138
139     END
```


5. 최적의 ARM 코드 탐색

4. Function#2 – Image negative [전체 수행 시간]

Address: 0x20000000

0x20000000:	87 A1 A3 88 8C DC 6D 13 E4 DE F7 4D 88 70 A7 98 10 4A 00 84 20 0C C3 4C 22 62 92 7D FD BF
0x20000001E:	98 C0 08 7F 4C E3 56 D5 6D CD B0 61 3D 3C 3C 06 0F 0E 4A 82 E8 CC 3A 2D 2E 49 92 14 CA 79
0x20000003C:	76 22 2F 23 B0 2E B2 CB A2 32 45 3A 93 4E 49 0F 4A 92 8B CE 39 D3 2E 38 32 CD 38 51 4E 49
0x20000005A:	4B 32 E7 24 B9 4B 77 34 EB 49 CE CB 4B 43 B8 39 2B 93 52 B4 3B CE 0B 44 25 AA 10 BB E8 A2
0x200000078:	B9 CB 01 10 E8 B2 30 74 92 C1 85 1A B2 EC 14 F9 80 92 D9 45 99 86 1A 68 27 5D 0A 0F 04 FC
0x200000096:	4E 39 29 CE B2 AC F2 60 2F 4C 0F 34 33 45 20 A0 76 86 5C 96 65 18 A6 80 D4 C0 94 79 B9 08
0x2000000B4:	32 E7 02 FA 93 CE 21 B2 1C 47 D1 A5 4E C0 BB 8C 33 A4 23 B9 93 BB C8 C8 92 EE B4 2E D0 A4
0x2000000D2:	32 ED 94 BA 2C 09 93 72 C1 84 1A 18 43 3B 27 4E D3 53 EC 3C CF 29 B1 2E 62 90 80 5D 1C 85
0x2000000F0:	4C 61 EC 2C E7 CE 92 33 0C FC 94 44 A5 E0 29 CA 65 04 05 89 74 A3 4A 11 20 15 20 28 0F B7
0x20000010E:	AA 10 2A B2 93 92 95 BA 2C 05 53 9A 8B DC 14 38 2F B1 39 19 18 96 4E 9C 78 2F 91 C2 A7 06
0x20000012C:	2A 4C 27 77 5E 92 24 4E 61 D3 28 D6 51 74 92 0B 85 C2 39 08 05 46 D0 14 0A 18 82 90 66 06
0x20000014A:	A9 04 90 69 A7 9C 79 1A 1F 5C F0 C1 84 89 0C 1E 69 87 41 38 01 0D 75 92 CE F9 82 60 34 61
0x200000168:	4C 09 04 08 86 61 18 1A 94 82 41 05 05 41 74 0A 0A 4D 14 B4 30 30 BC A2 20 1C C2 C9 BD A1
0x200000186:	45 4D 78 48 32 80 A3 1C E9 52 4E 4A 97 5D 72 66 5D 1C D3 C3 5C 08 34 C2 66 92 44 8A 0E F8
0x2000001A4:	05 4E 08 40 4A 59 78 49 14 1A FE 93 F9 92 77 71 C9 8D 4E 91 93 D2 DC C8 B9 2C 71 3B 5B 8A
0x2000001C2:	78 77 E7 C7 E3 88 C0 CB 0E 0F EB B0 F0 4E C0 3A 9E C3 01 9E E8 44 CE E1 04 E7 B1 C7 13 F0
0x2000001E0:	E5 61 1D C0 3A 16 75 8E C7 2F 72 25 04 09 D6 1E E5 A2 94 00 8A AE 84 90 6E 84 76 04 CA B0
0x2000001FE:	CA 5D A6 A6 30 EE CA 49 67 9E 06 A4 5A 2E 46 50 AC B3 E5 68 96 30 28 C9 18 C1 61 77 1A 45
0x20000021C:	43 0E 63 27 69 D1 0C 2A 08 C8 8E 21 0E 8D 86 67 38 67 22 7A 0C 38 8E 80 58 38 38 68 E0 81
0x20000023A:	F0 B0 F0 E3 44 87 73 EE A2 2F 8A 2E BC B3 D4 34 C1 A4 A2 50 85 C3 4C 49 5C 76 DE 23 44 94
0x200000258:	07 86 19 15 5C 74 87 2F 41 C4 10 B0 BC 3C 46 22 27 9D 18 37 A6 86 E0 13 09 24 55 A2 19 BA
0x200000276:	95 72 D1 99 6A 52 25 DE 5D 5C E4 05 49 C9 65 72 51 20 50 49 96 29 2F 41 8C 08 16 D2 08 70
0x200000294:	5A 13 F2 C0 16 C0 8A 12 BC 81 2D 82 46 10 06 42 09 81 26 80 21 85 02 65 D0 29 2F 19 73 51
0x2000002B2:	53 D4 80 A1 18 67 34 4E F1 60 D3 49 69 29 FB 48 26 A9 52 0C 93 A4 CB 63 39 84 60 AF 89 98
0x2000002D0:	29 67 46 11 CC 13 0F E6 18 41 29 03 E0 4E 52 8B 42 30 69 40 96 A7 A7 48 B2 71 72 61 86 F0
0x2000002EE:	4A 83 2F F4 69 9C 5D 66 4F 13 7F 33 82 80 69 67 29 94 14 B4 51 F3 64 09 28 3A 46 8B 32 B2

- 전체 수행시간 – 메모리 재배치 수행시간
= 이미지 변환 수행시간 = 0.00298678sec
- 이미지 변환 함수 수행시간은 약 3.5ms에서
약 2.98ms로 0.5ms 정도 단축됨

Registers

Register	Value
Current	
R0	0x41002580
R1	0x41004B00
R2	0x41007080
R3	0x20007080
R4	0x00000000
R5	0x000000DF
R6	0x0000004B
R7	0x000000ED
R8	0x00000000
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13 (SP)	0x00000000
R14 (LR)	0x00000008
R15 (PC)	0x00000138
CPSR	0x600000D3
SPSR	0x00000000
User/System	
Fast Interrupt	
Interrupt	
Supervisor	
Abort	
Undefined	
Internal	
PC \$	0x00000138
Mode	Supervisor
States	371223
Sec	0,00618705

[최적화 코드 전체 수행시간]

Registers

Register	Value
Current	
R0	0x40009636
R1	0x41002580
R2	0x41004B00
R3	0x41007080
R4	0x00000000
R5	0x00000012
R6	0x00000000
R7	0x00000000
R8	0x00000000
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13 (SP)	0x00000000
R14 (LR)	0x00000008
R15 (PC)	0x00000044
CPSR	0x600000D3
SPSR	0x00000000
User/System	
Fast Interrupt	
Interrupt	
Supervisor	
Abort	
Undefined	
Internal	
PC \$	0x00000044
Mode	Supervisor
States	192016
Sec	0,00320027

[최적화 코드 전체 메모리 재배치 시간]

5. 최적의 ARM 코드 탐색

4. Function#3 – Grayscale [기존 코드]

```
1      AREA Reset, CODE, READONLY
2      EXPORT start
3
4      start
5          BL relocate
6          BL re_grayscale
7
8      relocate
9          LDR    r0, =0x40000036
10         LDR    r1, =0x41000000
11         LDR    r2, =0x41002580
12         LDR    r3, =0x41004B00
13         MOV    r4, #9600
14
15     loop_relocate
16         LDRB    r5, [r0], #1
17         STRB    r5, [r1], #1
18
19         LDRB    r5, [r0], #1
20         STRB    r5, [r2], #1
21
22         LDRB    r5, [r0], #1
23         STRB    r5, [r3], #1
24
25         ADD    r0, r0, #1          ; A 무시
26         SUBS   r4, r4, #1
27         BNE    loop_relocate
28         BX     lr
```

```
30     re_grayscale
31         LDR    r0, =0x41000000    ; R
32         LDR    r1, =0x41002580    ; G
33         LDR    r2, =0x41004B00    ; B
34         LDR    r3, =0x20000000    ; Grayscale 저장 주소
35         MOV    r4, #9600          ; 픽셀 수
36
37     loop_gray
38         LDRB    r5, [r0], #1      ; R
39         LDRB    r6, [r1], #1      ; G
40         LDRB    r7, [r2], #1      ; B
41
42         ; grayscale = (3R + 6G + B) >> 3
43         ADD    r8, r5, r6, LSL #1
44         ADD    r8, r8, r8, LSL #1
45         ADD    r8, r8, r7
46
47         STRH    r8, [r3], #2      ; 1바이트로 저장
48
49         SUBS   r4, r4, #1
50         BNE    loop_gray
51
52     END
```

5. 최적의 ARM 코드 탐색

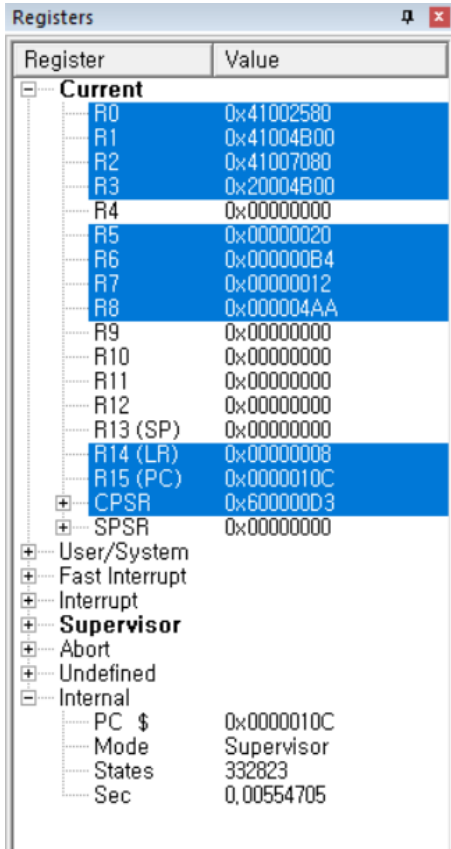
4. Function#3 – Grayscale [최적화 후 코드]

```
1      AREA Reset, CODE, READONLY
2      EXPORT start
3
4      start
5          BL relocate
6          BL re_grayscale
7
8      loop_end
9          B loop_end      ; ?? ??
10
11     relocate
12         LDR    r0, =0x40000036
13         LDR    r1, =0x41000000
14         LDR    r2, =0x41002580
15         LDR    r3, =0x41004B00
16         MOV    r4, #9600
17
18     loop_relocate
19         LDRB   r5, [r0], #1
20         STRB   r5, [r1], #1
21
22         LDRB   r5, [r0], #1
23         STRB   r5, [r2], #1
24
25         LDRB   r5, [r0], #1
26         STRB   r5, [r3], #1
27
28         ADD    r0, r0, #1      ; A ??
29         SUBS   r4, r4, #1
30         BNE    loop_relocate
31         BX     lr
```

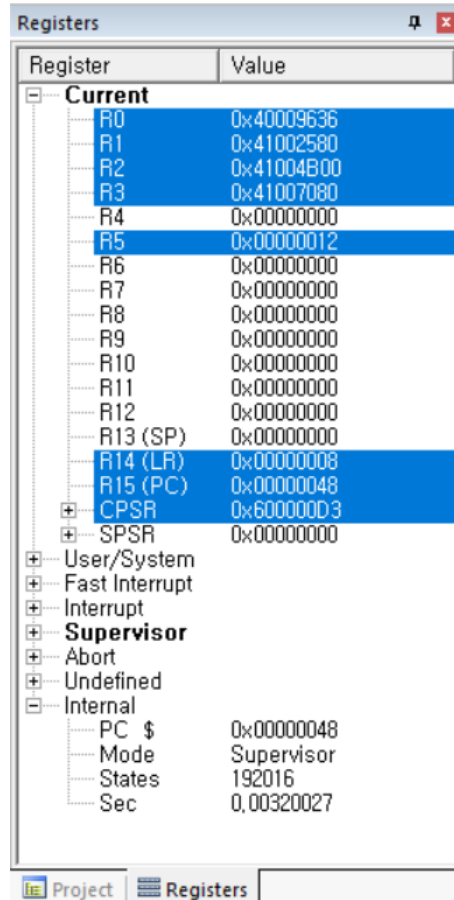
```
33     re_grayscale
34         LDR    r0, =0x41000000      ; R
35         LDR    r1, =0x41002580      ; G
36         LDR    r2, =0x41004B00      ; B
37         LDR    r3, =0x20000000      ; Grayscale ?? ??
38         MOV    r4, #1600            ; ?? ?
39
40     loop_gray
41         LDRB   r5, [r0], #1      ; R
42         LDRB   r6, [r1], #1      ; G
43         LDRB   r7, [r2], #1      ; B
44
45         ; grayscale = (3R + 6G + B) >> 3
46         ADD    r8, r5, r6, LSL #1
47         ADD    r8, r8, r8, LSL #1
48         ADD    r8, r8, r7
49
50         STRH   r8, [r3], #2      ; 1???? ??
51
52         LDRB   r5, [r0], #1      ; R
53         LDRB   r6, [r1], #1      ; G
54         LDRB   r7, [r2], #1      ; B
55
56         ; grayscale = (3R + 6G + B) >> 3
57         ADD    r8, r5, r6, LSL #1
58         ADD    r8, r8, r8, LSL #1
59         ADD    r8, r8, r7
60
61         STRH   r8, [r3], #2      ; 1???? ??
```

5. 최적의 ARM 코드 탐색

4. Function#3 – Grayscale [최적화 후 코드]



Register	Value
Current	
R0	0x41002580
R1	0x41004B00
R2	0x41007080
R3	0x20004B00
R4	0x00000000
R5	0x00000020
R6	0x000000B4
R7	0x00000012
R8	0x000004AA
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13 (SP)	0x00000000
R14 (LR)	0x00000008
R15 (PC)	0x0000010C
CPSR	0x600000D3
SPSR	0x00000000
User/System	
Fast Interrupt	
Interrupt	
Supervisor	
Abort	
Undefined	
Internal	
PC \$	0x0000010C
Mode	Supervisor
States	332823
Sec	0,00554705



Register	Value
Current	
R0	0x40009636
R1	0x41002580
R2	0x41004B00
R3	0x41007080
R4	0x00000000
R5	0x00000012
R6	0x00000000
R7	0x00000000
R8	0x00000000
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13 (SP)	0x00000000
R14 (LR)	0x00000008
R15 (PC)	0x00000048
CPSR	0x600000D3
SPSR	0x00000000
User/System	
Fast Interrupt	
Interrupt	
Supervisor	
Abort	
Undefined	
Internal	
PC \$	0x00000048
Mode	Supervisor
States	192016
Sec	0,00320027

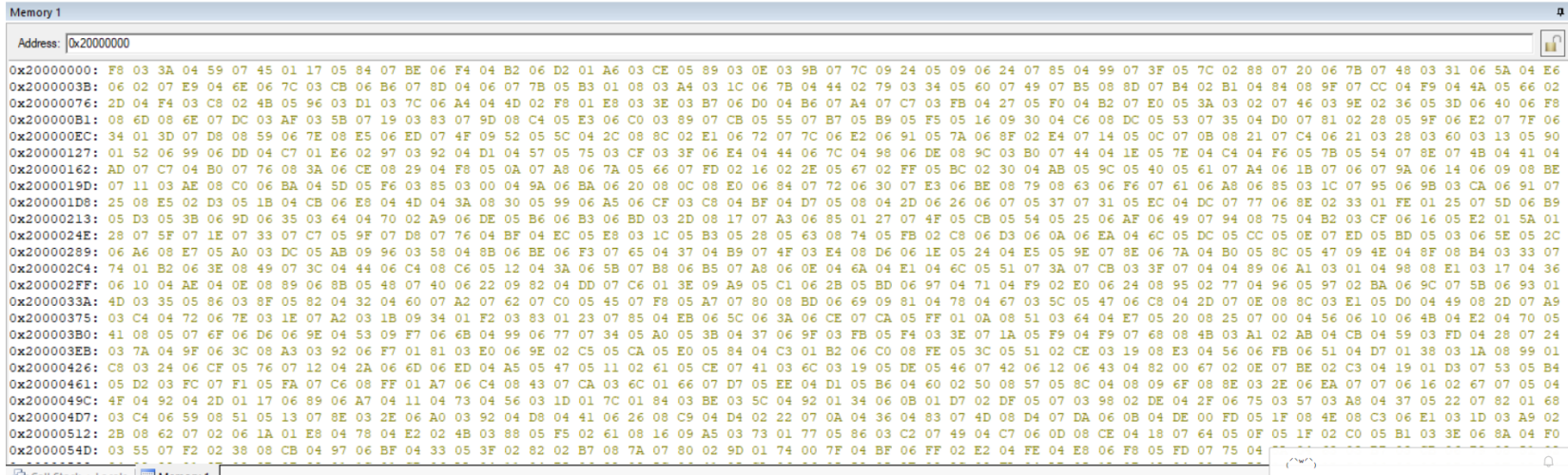
- 전체 수행시간 – 메모리 재배치 수행시간 = grayscale 변환 시간 = 0.00234678sec
- 이미지 변환 함수 수행시간은 약 3.5ms에서 약 2.98ms로 0.5ms 정도 단축됨

[최적화 코드 전체 수행시간]

[최적화 코드 전체 메모리 재배치 시간]

5. 최적의 ARM 코드 탐색

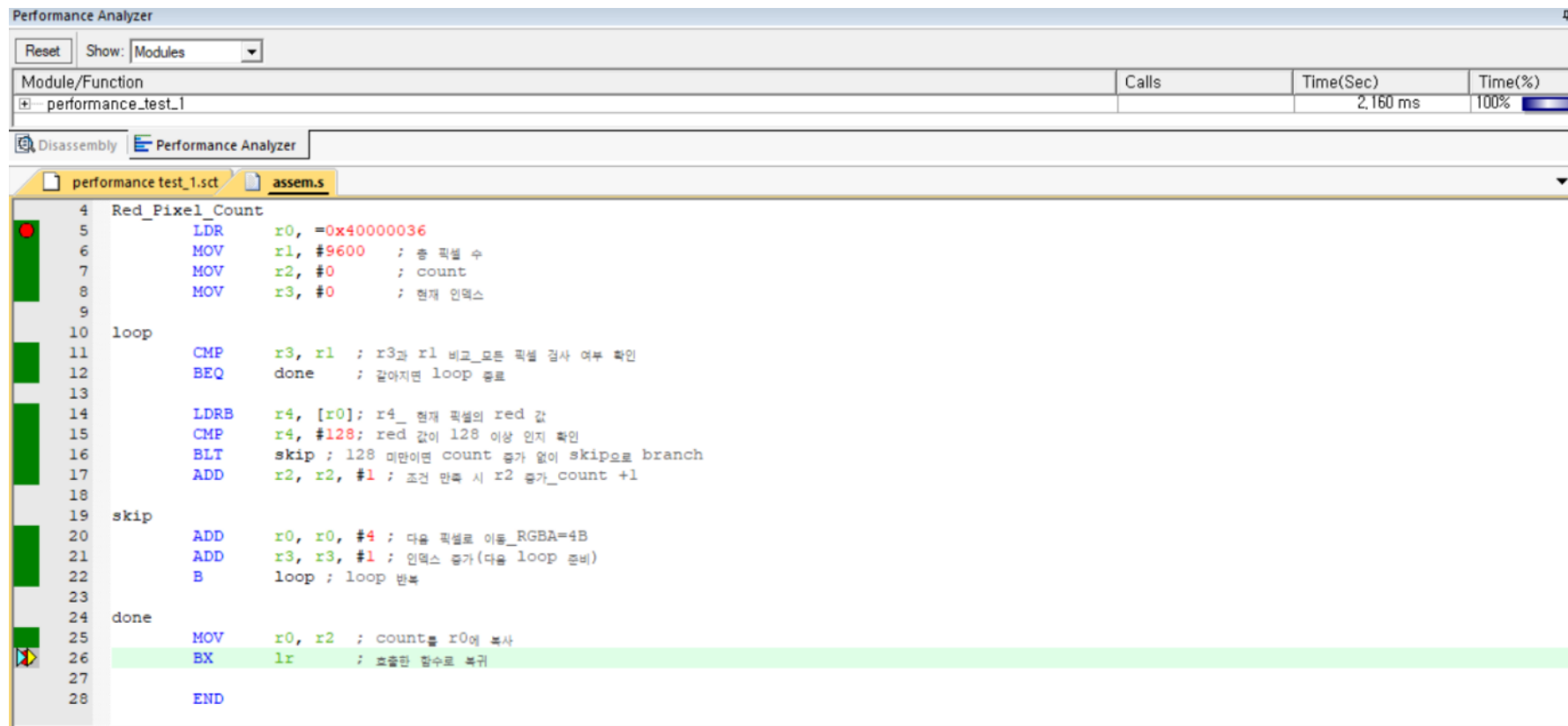
4. Function#3 – Grayscale [최적화 후 코드]



- 수행이 완료되면 start가 bl로 호출된 것이 아니므로 lr에 정상 리턴 주소가 없어 코드를 모두 실행한 후 쓰레기 값으로 점프하는 현상 >> loop end를 추가해 안정화
- 기존에 약 2.9ms의 수행시간에서 약 2.3ms로 약 0.6ms의 수행시간 단축

6. 3번과 4번의 사이의 ARM 코드 성능 차이 분석

- **Subproject#1:** 성능 비교에 Performance analyzer를 통해 함수 수행 시간 사용



The screenshot displays the Performance Analyzer interface. At the top, a table lists the performance of the function 'performance_test_1'.

Module/Function	Calls	Time(Sec)	Time(%)
performance_test_1		2,160 ms	100%

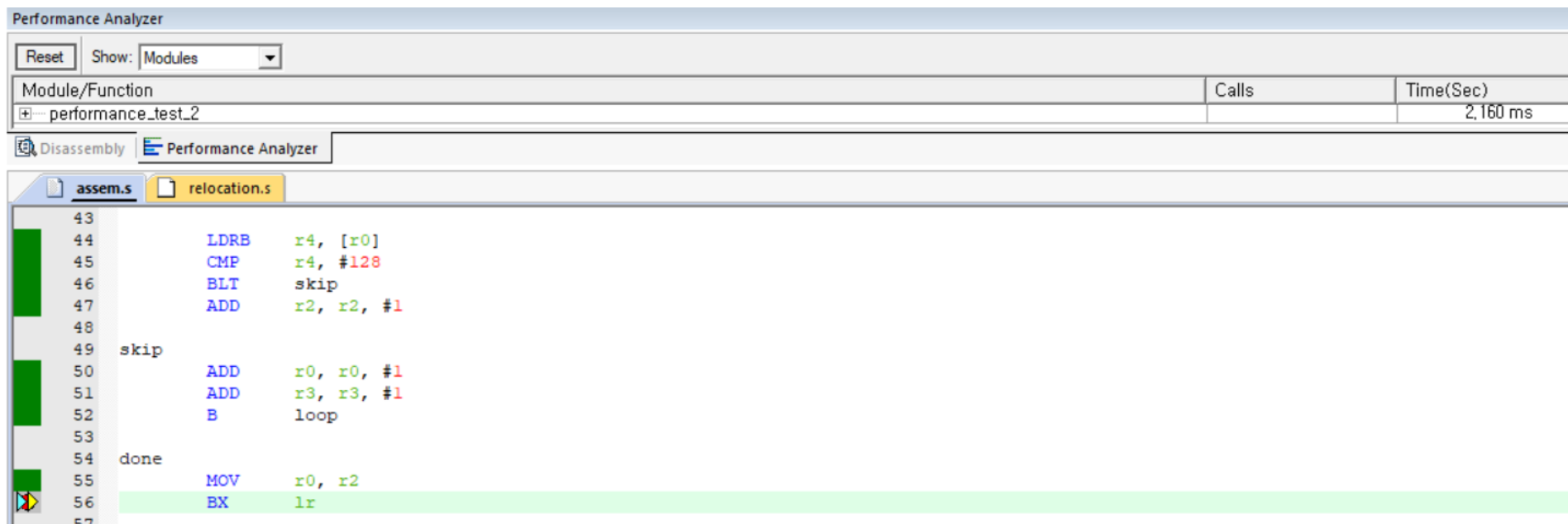
Below the table, the 'Performance Analyzer' tab is selected. The assembly code for 'performance_test_1.sct' is shown, with the 'assem.s' file open. The code is as follows:

```
4 Red_Pixel_Count
5     LDR    r0, =0x40000036
6     MOV    r1, #9600 ; 총 픽셀 수
7     MOV    r2, #0 ; count
8     MOV    r3, #0 ; 현재 인덱스
9
10 loop
11     CMP    r3, r1 ; r3과 r1 비교_모든 픽셀 검사 여부 확인
12     BEQ    done ; 같아지면 loop 종료
13
14     LDRB    r4, [r0]; r4_ 현재 픽셀의 red 값
15     CMP    r4, #128; red 값이 128 이상 인지 확인
16     BLT    skip ; 128 미만이면 count 증가 없이 skip으로 branch
17     ADD    r2, r2, #1 ; 조건 만족 시 r2 증가_count +1
18
19 skip
20     ADD    r0, r0, #4 ; 다음 픽셀로 이동_RGBA=4B
21     ADD    r3, r3, #1 ; 인덱스 증가(다음 loop 준비)
22     B      loop ; loop 반복
23
24 done
25     MOV    r0, r2 ; count를 r0에 복사
26     BX     lr ; 호출한 함수로 복귀
27
28     END
```

메모리 재배열 전 : 2.160 [ms]

6. 3번과 4번의 사이의 ARM 코드 성능 차이 분석

- **Subproject#1:** 성능 비교에 Performance analyzer를 통해 함수 수행 시간 사용



메모리 재배열 후 : 2.160 [ms]

6. 3번과 4번의 사이의 ARM 코드 성능 차이 분석

▪ Subproject#1

▪ 메모리 재배열과 캐시 효율성

- 기존 구조: RGBA가 4바이트 단위로 순차 저장
 - > CPU 캐시의 spatial locality를 제대로 활용하지 못함
- 재배열 이후: R, G, B 각각 연속된 메모리 영역에 분리 저장
 - > 동일한 채널 데이터에 대한 집중 접근 가능
 - > 캐시 miss 감소, 메모리 접근 횟수 감소
- 이론적 기대
 - > 예측 가능한 메모리 접근 패턴으로 함수 전체 수행 시간 감소 예상

• 실험 결과와 시사점

- 실제 실행 시간 개선은 미미하거나 측정 불가능한 수준
- 예상 원인: 성능 차이가 너무 작아 측정이 어려움 or 비효율적인 코드 구조로 인해 최적화 효과 미반영
- 결론: 성능 향상을 위해 명령어 수 조절, 루프 구조 최적화 등 추가 작업 필요

6. 3번과 4번의 사이의 ARM 코드 성능 차이 분석

▪ Subproject#2: 성능 비교표

항목	원본 방식	리로케이션 방식
루프 수	1 (9600회)	1(9600회)
명령어 수 / 픽셀	약 7	약 12~14
메모리 접근 수 / 픽셀	7 (4 read + 3 write)	13 (6 read + 7 write)
총 명령 실행 수	약 67,200	약 124,800 이상
코드 구조	단순	모듈화, 재사용 가능
캐시 효율	높음 (연속 접근)	낮음 (분리 접근)
병렬성 / 확장성	낮음	높음

6. 3번과 4번의 사이의 ARM 코드 성능 차이 분석

■ Subproject#2: 성능 비교표

[메모리 재배치 전 수행시간]

Internal
PC \$ 0x0000003C
Mode Supervisor
States 220801
Sec 0,00368002

Internal
PC \$ 0x00000084
Mode Supervisor
States 403223
Sec 0,00672038

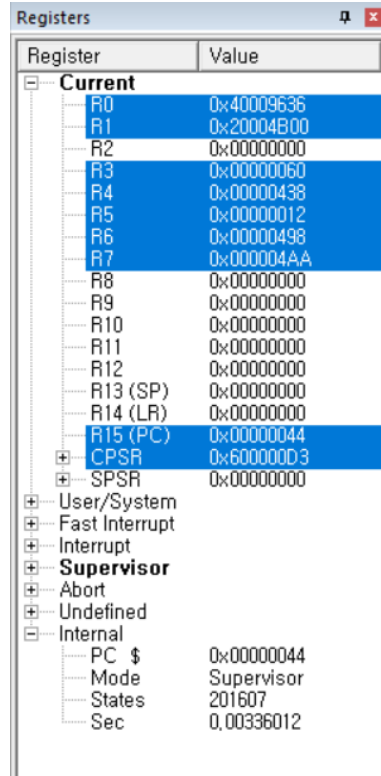
Internal
PC \$ 0x00000044
Mode Supervisor
States 192016
Sec 0,00320027

[메모리 재배치 후 전체 코드 수행시간 & 재배치만 하는데 걸린 시간]

- 전체 수행 시간 - 재배치 시간 = 이미지 변환 함수 수행 시간 = 0.00352011 sec

6. 3번과 4번의 사이의 ARM 코드 성능 차이 분석

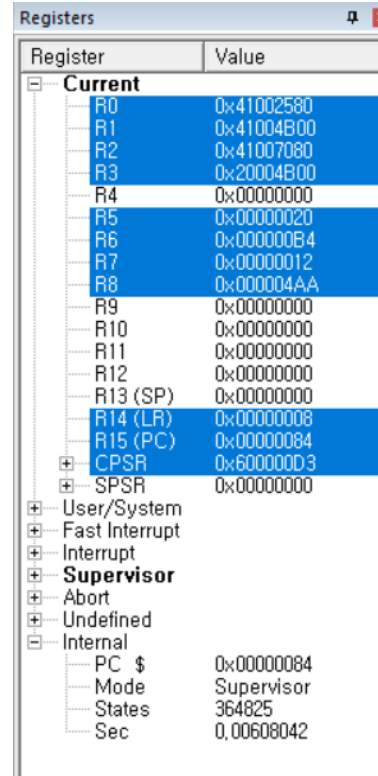
■ Subproject#3:



Registers window showing the state of registers before memory allocation. The 'Current' section shows R0 through R15 and CPSR. R15 (PC) is 0x00000044. The 'Supervisor' section shows Abort, Undefined, and Internal states. The 'Internal' section shows PC \$, Mode, States, and Sec.

Register	Value
R0	0x40009636
R1	0x20004B00
R2	0x00000000
R3	0x00000060
R4	0x00000438
R5	0x00000012
R6	0x00000498
R7	0x000004AA
R8	0x00000000
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13 (SP)	0x00000000
R14 (LR)	0x00000000
R15 (PC)	0x00000044
CPSR	0x60000003
SPSR	0x00000000
User/System	
Fast Interrupt	
Interrupt	
Supervisor	
Abort	
Undefined	
Internal	
PC \$	0x00000044
Mode	Supervisor
States	201607
Sec	0.00336012

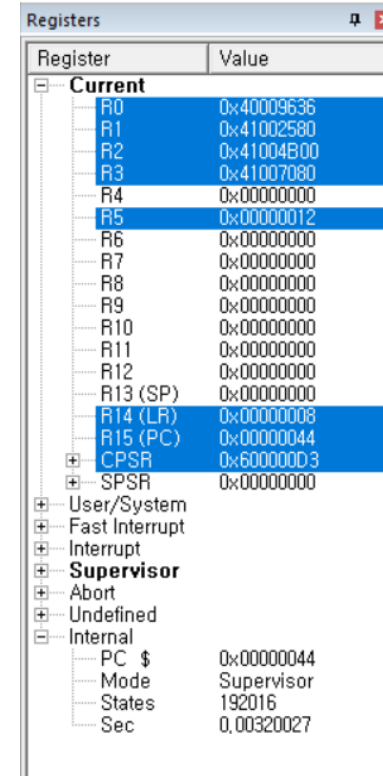
[메모리 재배열 전]
수행시간 = 0.00336012sec



Registers window showing the state of registers during total execution. The 'Current' section shows R0 through R15 and CPSR. R15 (PC) is 0x00000084. The 'Supervisor' section shows Abort, Undefined, and Internal states. The 'Internal' section shows PC \$, Mode, States, and Sec.

Register	Value
R0	0x41002580
R1	0x41004B00
R2	0x41007080
R3	0x20004B00
R4	0x00000000
R5	0x00000020
R6	0x000000B4
R7	0x00000012
R8	0x000004AA
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13 (SP)	0x00000000
R14 (LR)	0x00000008
R15 (PC)	0x00000084
CPSR	0x60000003
SPSR	0x00000000
User/System	
Fast Interrupt	
Interrupt	
Supervisor	
Abort	
Undefined	
Internal	
PC \$	0x00000084
Mode	Supervisor
States	364825
Sec	0.00608042

[전체코드 수행시간]



Registers window showing the state of registers during memory allocation. The 'Current' section shows R0 through R15 and CPSR. R15 (PC) is 0x00000044. The 'Supervisor' section shows Abort, Undefined, and Internal states. The 'Internal' section shows PC \$, Mode, States, and Sec.

Register	Value
R0	0x40009636
R1	0x41002580
R2	0x41004B00
R3	0x41007080
R4	0x00000000
R5	0x00000012
R6	0x00000000
R7	0x00000000
R8	0x00000000
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13 (SP)	0x00000000
R14 (LR)	0x00000008
R15 (PC)	0x00000044
CPSR	0x60000003
SPSR	0x00000000
User/System	
Fast Interrupt	
Interrupt	
Supervisor	
Abort	
Undefined	
Internal	
PC \$	0x00000044
Mode	Supervisor
States	192016
Sec	0.00320027

[메모리 재배치 수행시간]

- 그레이스케일 변환 시간 = 전체 수행시간 - 메모리 재배치 수행시간 = 0.00288015sec

7. 결과물 (최종 코드)

■ Subproject#1:

```
1      AREA    RelocateAndCount, CODE, READONLY
2      EXPORT  relocate_and_count_red
3      ENTRY
4
5  relocate_and_count_red
6      PUSH    {R4-R9, LR}
7
8      MOV     R1, #0
9      LDR     R2, =0x40000036      ; RGBA ?? ??
10     LDR     R3, =0x50000000      ; R ???
11     LDR     R4, =0x50002580      ; G ???
12     LDR     R5, =0x50004B00      ; B ???
13
14 ; == [1??] RGBA -> RRR... GGG... BBB... ?? ?? ==
15 relocate_loop
16     CMP     R1, #9600
17     BGE     count_start
18
19     LDRB    R6, [R2]              ; R
20     LDRB    R7, [R2, #1]          ; G
21     LDRB    R8, [R2, #2]          ; B
22
23     STRB    R6, [R3], #1          ; R ??
24     STRB    R7, [R4], #1          ; G ??
25     STRB    R8, [R5], #1          ; B ??
26
27     ADD     R2, R2, #4
28     ADD     R1, R1, #1
29     B       relocate_loop
```

```
32 count_start
33     MOV     R0, #0
34     MOV     R1, #1600
35     LDR     R3, =0x50000000
36
37 count_loop
38     LDRB    R6, [R3], #1
39     CMP     R6, #128
40     ADDGE   R0, R0, #1
41
42     LDRB    R6, [R3], #1
43     CMP     R6, #128
44     ADDGE   R0, R0, #1
45
46     LDRB    R6, [R3], #1
47     CMP     R6, #128
48     ADDGE   R0, R0, #1
49
50     LDRB    R6, [R3], #1
51     CMP     R6, #128
52     ADDGE   R0, R0, #1
53
54     LDRB    R6, [R3], #1
55     CMP     R6, #128
56     ADDGE   R0, R0, #1
57
58     LDRB    R6, [R3], #1
59     CMP     R6, #128
60     ADDGE   R0, R0, #1
61
62
63     SUBS    R1, R1, #1
64     BNE     count_loop
65
66 done
67     POP     {R4-R9, PC}
68
69     END
70
```


7. 결과물 (최종 코드)

■ Subproject#2:

```
1      AREA Reset, CODE, READONLY
2      EXPORT start
3
4      start
5          BL relocate
6          BL re_rgb_negative
7
8      relocate
9          LDR    r0, =0x40000036
10         LDR    r1, =0x41000000
11         LDR    r2, =0x41002580
12         LDR    r3, =0x41004B00
13         MOV    r4, #9600
14
15     loop_relocate
16         LDRB    r5, [r0], #1
17         STRB    r5, [r1], #1
18
19         LDRB    r5, [r0], #1
20         STRB    r5, [r2], #1
21
22         LDRB    r5, [r0], #1
23         STRB    r5, [r3], #1
24
25         ADD    r0, r0, #1
26         SUBS    r4, r4, #1
27         BNE    loop_relocate
28         BX     lr
29
```

```
30     re_rgb_negative
31         LDR    r0, =0x41000000
32         LDR    r1, =0x41002580
33         LDR    r2, =0x41004B00
34         LDR    r3, =0x20000000
35         MOV    r4, #1600
36
37     loop_negative
38         LDRB    r5, [r0], #1
39         RSB     r5, r5, #255
40
41
42         LDRB    r6, [r1], #1
43         RSB     r6, r6, #255
44
45
46         LDRB    r7, [r2], #1
47         RSB     r7, r7, #255
48
49
50         STRB    r5, [r3], #1
51         STRB    r6, [r3], #1
52         STRB    r7, [r3], #1
53
```

7. 결과물 (최종 코드)

■ Subproject#2:

```
54      LDRB    r5, [r0], #1
55      RSB     r5, r5, #255
56
57
58      LDRB    r6, [r1], #1
59      RSB     r6, r6, #255
60
61
62      LDRB    r7, [r2], #1
63      RSB     r7, r7, #255
64
65
66      STRB    r5, [r3], #1
67      STRB    r6, [r3], #1
68      STRB    r7, [r3], #1
69
70      LDRB    r5, [r0], #1
71      RSB     r5, r5, #255
72
73
74      LDRB    r6, [r1], #1
75      RSB     r6, r6, #255
76
77
78      LDRB    r7, [r2], #1
79      RSB     r7, r7, #255
80
81
82      STRB    r5, [r3], #1
83      STRB    r6, [r3], #1
84      STRB    r7, [r3], #1
85
```

```
118      LDRB    r5, [r0], #1
119      RSB     r5, r5, #255
120
121
122      LDRB    r6, [r1], #1
123      RSB     r6, r6, #255
124
125
126      LDRB    r7, [r2], #1
127      RSB     r7, r7, #255
128
129
130      STRB    r5, [r3], #1
131      STRB    r6, [r3], #1
132      STRB    r7, [r3], #1
133
134      SUBS    r4, r4, #1
135      BNE     loop_negative
136
137      BX      lr
138
139      END
```

7. 결과물 (최종 코드)

■ Subproject#3:

```
1      AREA Reset, CODE, READONLY
2      EXPORT start
3
4  start
5      BL relocate
6      BL re_grayscale
7
8  loop_end
9      B loop_end      ; ???
10
11  relocate
12      LDR    r0, =0x40000036
13      LDR    r1, =0x41000000
14      LDR    r2, =0x41002580
15      LDR    r3, =0x41004B00
16      MOV    r4, #9600
17
18  loop_relocate
19      LDRB   r5, [r0], #1
20      STRB   r5, [r1], #1
21
22      LDRB   r5, [r0], #1
23      STRB   r5, [r2], #1
24
25      LDRB   r5, [r0], #1
26      STRB   r5, [r3], #1
27
28      ADD    r0, r0, #1      ; A ??
29      SUBS   r4, r4, #1
30      BNE    loop_relocate
31      BX     lr
```

```
33  re_grayscale
34      LDR    r0, =0x41000000      ; R
35      LDR    r1, =0x41002580      ; G
36      LDR    r2, =0x41004B00      ; B
37      LDR    r3, =0x20000000      ; Grayscale ?? ??
38      MOV    r4, #1600            ; ???
39
40  loop_gray
41      LDRB   r5, [r0], #1      ; R
42      LDRB   r6, [r1], #1      ; G
43      LDRB   r7, [r2], #1      ; B
44
45      ; grayscale = (3R + 6G + B) >> 3
46      ADD    r8, r5, r6, LSL #1
47      ADD    r8, r8, r8, LSL #1
48      ADD    r8, r8, r7
49
50      STRH   r8, [r3], #2      ; 1???? ??
51
52      LDRB   r5, [r0], #1      ; R
53      LDRB   r6, [r1], #1      ; G
54      LDRB   r7, [r2], #1      ; B
55
56      ; grayscale = (3R + 6G + B) >> 3
57      ADD    r8, r5, r6, LSL #1
58      ADD    r8, r8, r8, LSL #1
59      ADD    r8, r8, r7
60
61      STRH   r8, [r3], #2      ; 1???? ??
62
```

7. 결과물 (최종 코드)

■ Subproject#3:

```
63      LDRB    r5, [r0], #1      ; R
64      LDRB    r6, [r1], #1      ; G
65      LDRB    r7, [r2], #1      ; B
66
67      ; grayscale = (3R + 6G + B) >> 3
68      ADD     r8, r5, r6, LSL #1
69      ADD     r8, r8, r8, LSL #1
70      ADD     r8, r8, r7
71
72      STRH    r8, [r3], #2      ; 1???? ??
73
74
75
76      LDRB    r5, [r0], #1      ; R
77      LDRB    r6, [r1], #1      ; G
78      LDRB    r7, [r2], #1      ; B
79
80      ; grayscale = (3R + 6G + B) >> 3
81      ADD     r8, r5, r6, LSL #1
82      ADD     r8, r8, r8, LSL #1
83      ADD     r8, r8, r7
84
85      STRH    r8, [r3], #2
```

```
87      LDRB    r5, [r0], #1      ; R
88      LDRB    r6, [r1], #1      ; G
89      LDRB    r7, [r2], #1      ; B
90
91      ; grayscale = (3R + 6G + B) >> 3
92      ADD     r8, r5, r6, LSL #1
93      ADD     r8, r8, r8, LSL #1
94      ADD     r8, r8, r7
95
96      STRH    r8, [r3], #2
97
98      LDRB    r5, [r0], #1      ; R
99      LDRB    r6, [r1], #1      ; G
100     LDRB    r7, [r2], #1      ; B
101
102     ; grayscale = (3R + 6G + B) >> 3
103     ADD     r8, r5, r6, LSL #1
104     ADD     r8, r8, r8, LSL #1
105     ADD     r8, r8, r7
106
107     STRH    r8, [r3], #2
108
109     SUBS    r4, r4, #1
110     BNE     loop_gray
111
112     B loop_end
113
114     END
```

8. 프로젝트 수행일지

이름	유지원	김선영	박대호	최은지
모임일자	수행내용			
5/15	-프로젝트에 대해 논의 -각자 수행할 역할 분담 1): 최은지-Function1, 3에 대한 코드 작성, Function1 보고서 작성 2): 유지원-Function2에 대한 코드 작성, Function2 보고서 작성, ppt 제작 3): 김선영-Function3 보고서 작성, 발표, 수행일지 작성 4): 박대호-개요, 최적화 코드 작성, 보고서 취합			
5/29	-assembly 언어 27-bit RGB, 16-bit grayscale, pixel number 함수 작성 -assembly 언어 relocation 함수 작성 -relocation 과 3가지 function 프로젝트 구현			
5/31	-각자 작성한 보고서 공유, 피드백 후 취합			
6/4	-최종 보고서, ppt 제작			
6/7	-발표 대본 작성 -발표 영상 녹화			

9. Troubleshooting

■ Subproject#1

■ 디버깅 중 발생한 문제와 원인 분석

- 초기 구현 방식: C 코드와 어셈블리 코드를 연동
 - > Red 값 ≥ 128 인 픽셀 수를 세어 UART로 출력
- 문제 발생: 항상 **결과값 4627이 반복 출력**
- 원인 분석: Keil MDK의 기본 초기화 루틴인 startup.o 파일의 __scatterload 함수가 .bss, .data 뿐 아니라 .hex 파일로 메모리에 로드한 RGBA 이미지 데이터까지 초기화함

```
0x40000410: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x4000042A: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x40000444: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x4000045E: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x40000478: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 66 98 BE F4 3B 6D 74 DA 9F 7A
0x40000492: 36 F8 31 D9 10 1E A1 F7 5B DB F4 B1 CE 57 DB 47 19 D6 7C 13 1D CC 35 7D 6C AB
0x400004AC: DF C3 97 CA 2E 71 4D EC DA DB FE E3 48 B1 F6 3C 2B F6 24 7B 64 8E A5 C7 36 B5
0x400004C6: B1 6E 6F 3E 13 43 17 1C 23 16 6D 68 7B EF 1D 6E B9 BD C6 3C 71 DD 9D FC F3 B2
```

```
0x0000015C E24FE018 SUB R14,PC,#0x00000018
0x00000160 E3130001 TST R3,#0x00000001
0x00000164 1047F003 SUBNE PC,R7,R3
0x00000168 E12FFF13 BX R3
0x0000016C 000005B8 (???)EQ
0x00000170 000005C8 ANDEQ R0,R0,R8,ASR #11
        __scatterload_zeroinit:
0x00000174 E3B03000 MOVS R3,#0x00000000
0x00000178 E3B04000 MOVS R4,#0x00000000
0x0000017C E3B05000 MOVS R5,#0x00000000
0x00000180 E3B06000 MOVS R6,#0x00000000
0x00000184 E2522010 SUBS R2,R2,#0x00000010
0x00000188 28A10078 STMCSPA R1!,(R3-R6)
0x0000018C 8AFFFFFC BHI 0x00000184
0x00000190 E1B02E32 MOVS R2,R2,LSL #29
0x00000194 28A10030 STMCSPA R1!,(R4-R5)
0x00000198 45813000 STRMI R3,[R1]
```

```
AREA RESET, CODE, READONLY
ENTRY
EXPORT __main
IMPORT main

__main
    BL    main
    B     .

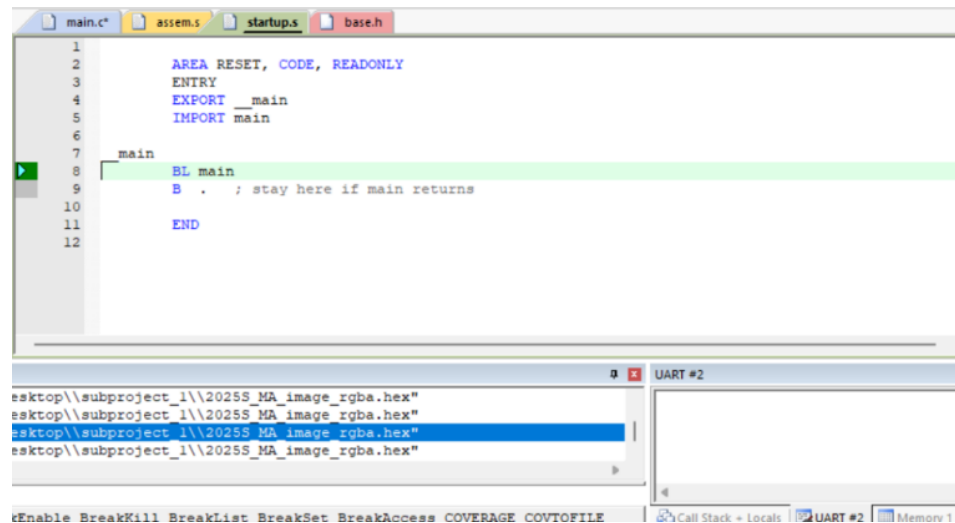
END
```

9. Troubleshooting

■ Subproject#1

■ 문제 해결을 위한 구조 수정

- 조치 내용: Keil의 기본 startup.o 코드를 제거하고 직접 작성한 startup.s를 사용해 main 함수만 호출하는 최소한의 초기화 구조로 변경
- 새로운 문제 발생: init_serial() 함수가 호출되지 않아 UART 출력이 전혀 동작하지 않음
- 원인: 기존 startup.o 내 초기화 루틴에서 수행되던 UART 관련 초기화 코드가 누락



```
1      AREA RESET, CODE, READONLY
2      ENTRY
3      EXPORT __main
4      IMPORT main
5      IMPORT init_serial
6
7      __main
8      BL    init_serial
9      BL    main
10     B     .
11
12     END
13
```


9. Troubleshooting

▪ Subproject#1

▪ UART 출력 문제 해결

- 조치 내용: Startup.s 파일에 IMPORT init_serial, BL init_serial을 추가하여 초기화 보완
- 결과: main 함수 진입 전에 UART 설정이 정상적으로 수행됨
 - > UART 출력 문제 해결, 디버깅 가능 상태 복원
- 의의: 필요한 초기화만 수동으로 지정하여 불필요한 초기화는 방지 + 필수기능은 유지

UART #2

```
Number of pixels_red valued(relocation) >= 128: 4803
```

▪ 구현 방식 변경 – Assembly 단독 구현

- 변경 이유: UART 출력은 정상 작동 but 코드 일관성, 잠재적 문제 방지 필요
 - > C코드 완전 제거, assembly 코드만으로 구현 전환
- 새로운 방식: 조건을 만족하는 픽셀 수를 계산한 후 결과값이 저장된 레지스터 r0의 16진수 값을 직접 확인 >> 이를 10진수로 변환하여 결과 검증
- 의의: 불필요한 의존성을 줄이고 코드 구조를 간결하게 통일

9. Troubleshooting

- Subproject#1

- 이를 통해 정확한 결과값인 4803 확인 & 문제의 근본 원인을 시스템 초기화 코드로부터 찾아내고 해결

Register	Value
Current	
R0	0x000012C3
R1	0x00002580
R2	0x000012C3
R3	0x00002580
R4	0x00000020
R5	0x00000000
R6	0x00000000
R7	0x00000000
R8	0x00000000
R9	0x00000000
R10	0x00000000
R11	0x00000000
R12	0x00000000
R13 (SP)	0x00000000
R14 (LR)	0x00000008
R15 (PC)	0x00000064
CPSR	0x600000D3
SPSR	0x00000000

9. Troubleshooting

▪ Subproject#2

- **error: L6235E: More than one section matches selector - cannot all be FIRST/LAST**
- **문제 원인 : 엔트리 포인트 중복**
 - 프로젝트에 `assem.s` 외에도 `startup.o`, `serial.o` 등이 링크 대상으로 포함되어 있었음
 - >> 이 파일들 중 일부가 이미 `RESET`, `+First` 등의 엔트리 포인트를 정의하고 있었음
 - >> 그 결과 링커가 진입점을 중복 인식해 충돌 발생
- **문제 해결 : 진입점 명확화 및 불필요 파일 제거**
 - `assem.s` 내부에서 `ENTRY`, `EXPORT` 명령어만 명확히 지정
 - 동시에 프로젝트 구성에서 `startup.o`, `serial.o` 파일 완전히 제거
 - 중복 엔트리 포인트 제거 -> 링커 충돌 문제 해결 완료

9. Troubleshooting

- Subproject#3

- 함수 구현 과정 – C -> Assembly 전환

- 초기 시도: 32-bit RGBA -> 16-bit Grayscale 변환 함수를 C 언어로 구현
 - > 시뮬레이션 정상 작동 실패
 - 조치: 함수를 Assembly 언어로 재작성
 - > 정확한 결과 및 안전성 확보
 - 최적화 시도: ARM 최적화 함수도 C로 작성했으나 동일 문제 발생
 - > 최종적으로 모든 이미지 처리 함수를 Assembly로 구현
 - C -> Assembly 전환이 프로젝트 성과 도출의 핵심

9. Troubleshooting

■ Subproject#3

■ 추가 최적화 시도 – Thumb 모드 실험

- Thumb 모드 실험 목적: 코드 크기 절감 및 실행 효율 향상
- 문제점: Thumb-1 명령어 셋 제한: R0~R7만 사용 가능
 - > 중간 계산 처리에 제약
 - > 디버깅 중 프로그램 비정상 종료, 루프 수행 중단 반복
- 결론: 실습의 안전성과 확장성 확보 어려움
 - > 최종적으로 ARM 모드 기반 최적화로 마무리

• 메모리 맵 설정 문제와 해결

- 초기 문제: Keil이 사용하는 내부 메모리와 실습용 RGBA 이미지 주소가 중복 등록됨
- 문제 발생 사례: 디버거 메모리 영역 오염, 실행 중 데이터가 예기치 않게 변경됨
- 해결 방안: 사용하지 않는 영역으로 명확히 분리 등록
 - > .sct 파일에서 READWRITE 권한 명시 -> 안정적 디버깅 환경 구축 성공



The First University in Korea

Soongsil University